

Table of Contents

[DSSCDefTopic](#)

[dsdsscdis.4.1](#)

[dsdsscdis.4.2](#)

[dsdsscdis.4.3](#)

[dsdsscov.1.2](#)

[dsdsscov.1.3](#)

[dsdsscov.1.4](#)

[dsdsscprog.3.01](#)

[dsdsscprog.3.02](#)

[dsdsscprog.3.03](#)

[dsdsscprog.3.04](#)

[dsdsscprog.3.05](#)

[dsdsscprog.3.06](#)

[dsdsscprog.3.07](#)

[dsdsscprog.3.08](#)

[dsdsscprog.3.09](#)

[dsdsscprog.3.10](#)

[dsdsscprog.3.11](#)

[dsdsscprog.3.12](#)

[dsdsscprog.3.13](#)

[dsdsscprog.3.14](#)

[dsdsscprog.3.15](#)

[dsdsscprog.3.16](#)

[dsdsscprog.3.17](#)

[dsdsscprog.3.18](#)

[dsdsscprog.3.19](#)

[dsdsscprog.3.20](#)

[dsdsscprog.3.21](#)

[dsdsscprog.3.22](#)

[dsdsscprog.3.23](#)

[dsdsscprog.3.24](#)

[dsdsscprog.3.25](#)

[dsdssctoo.2.1](#)

[dsdssctoo.2.2](#)

[dsdssctoo.2.3](#)

[dsdssctoo.2.4](#)

[dsdssctoo.2.5](#)

[dsdssctoo.2.6](#)

[dsdssctoo.2.7](#)

[dsscexamples.5.01](#)

[dsscexamples.5.02](#)

[dsscexamples.5.03](#)

[dsscexamples.5.04](#)

[dsscexamples.5.05](#)

[dsscexamples.5.06](#)

[dsscexamples.5.07](#)

[dsscexamples.5.08](#)

[dsscexamples.5.09](#)

[dsscexamples.5.10](#)

[dsscexamples.5.11](#)

[dsscexamples.5.12](#)

[dsscexamples.5.13](#)

[dsscexamples.5.14](#)

[dsscexamples.5.15](#)

[dsscexamples.5.16](#)

[dsscexamples.5.17](#)

DSSCDefTopic

<

>

About the DSSC

About the DSSC

Digital Signal Source and Capture (DSSC) is an instrument on each UltraFLEX HSD1000, UltraPin800, and UltraPin1600 digital channel card that:

- Sources high speed digital waveform representations for converter testing
- Sources dynamically generated digital data for device register writes
- Captures and stores to memory high speed digital waveform representations for converter testing
- Captures and stores to memory digital data from device register reads
- Sends captured data using instrument initiated moves (IIMs) to DSP resources for data processing
- Allows data to be independent from a digital pattern

This section includes the following topics:

- Features
- Safety Information
- DSSC Software Licensing Requirements

DSSC information includes the following sections:

- [DSSC Theory of Operation](#)

- [DSSC Programming](#)

- [DSSC Debug Display](#)

- [DSSC Examples](#)

Related Topics

- [HSD1000 hardware specifications:](#)

- [Digital Signal Source \(DigSrc\)](#)

- [Digital Signal Capture \(DigCap\)](#)

- [UltraPin800 hardware specifications:](#)

- [Digital Signal Source \(DigSrc\)](#)

- [Digital Signal Capture \(DigCap\)](#)

- [UltraPin1600 hardware specifications](#)

- [Digital Signal Source \(DigSrc\)](#)

- [Digital Signal Capture \(DigCap\)](#)

- [DSSC \(VBT syntax\)](#)

- [About the HSD1000](#)

- [HSD1000 DIB Design Guidelines](#)
-
- [About the UltraPin800](#)
-
- [UltraPin800 DIB Design Guidelines](#)
-
- [About the UltraPin1600](#)
-
- [UltraPin1600 DIB Design Guidelines](#)

dsdsscdis.4.1

<	>
-----------------	-----------------

DSSC Debug Display

DSSC Debug Display

The DSSC debug display is an interactive display that shows:

- A block diagram of the digital source or capture instrument with the stored settings of the digital source or digital capture instrument
- ADS lines used and connections from the digital source and capture to the digital pins

The display works in either off-line (workstation) or on-line (tester) mode. You can change values on the display and run your test program to see the effects of the change.

This section includes the following topics:

- Starting the DSSC Debug Display
- DSSC Debug Display Interface

This section describes features specific to the Digital Source and Capture debug display. For more information on using debug displays, see the following:

- Using Debug Displays in TDE
- Producing Code from Debug Displays

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsdsscdis.4.2

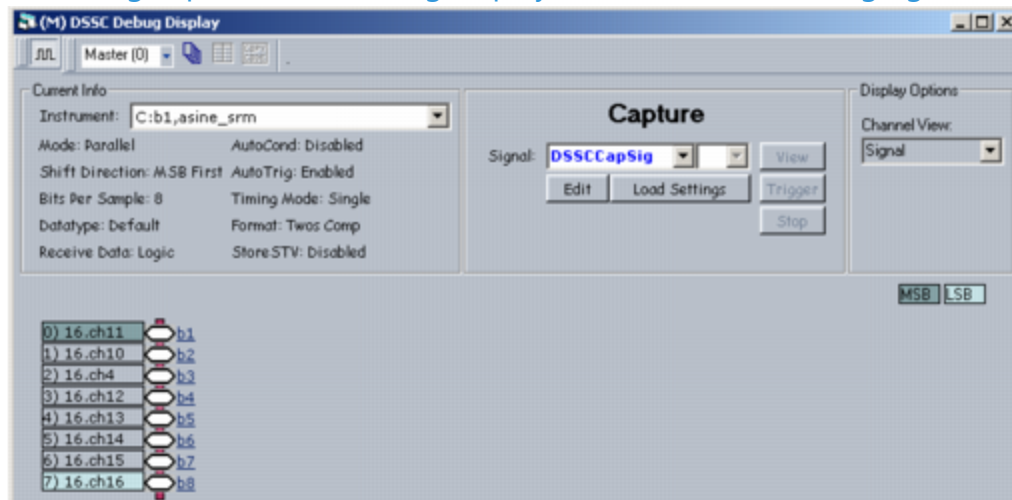


DSSC Debug Display : Starting the DSSC Debug Display

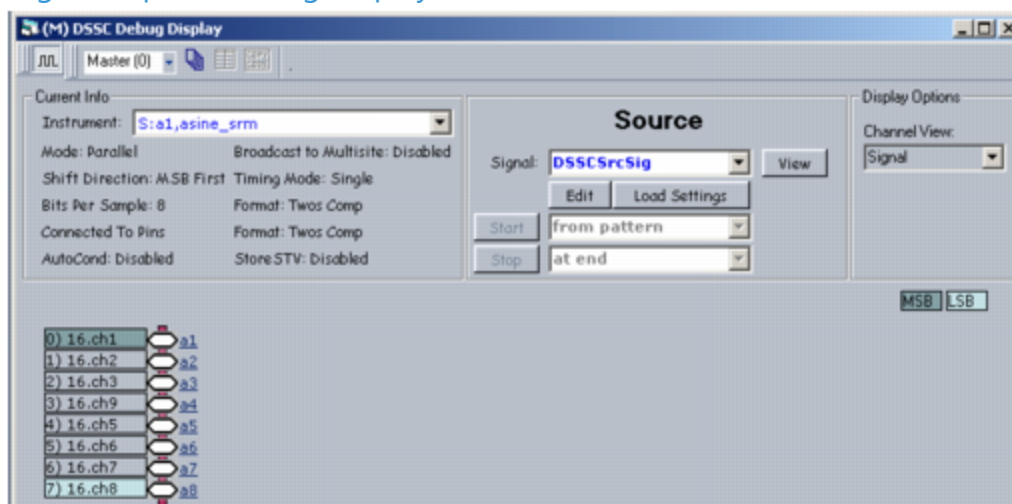
Starting the DSSC Debug Display

To bring up the DSSC display, on the IG-XL Tools tab, in the Launch group, select Debug Displays>DSSC.

This brings up the DSSC debug display shown in the following figures.



Digital Capture Debug Display



Digital Source Debug Display

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsdsscdis.4.3

<

>

DSSC Debug Display : DSSC Debug Display Interface

DSSC Debug Display Interface

This display has three main components:

Current Info panel	Describes all parameters of the currently selected instruments.
Display Options panel	Controls the general look of the debug display and the format of the information it shows.
Pins Display panel	Describes each bit in the instrument and its connection to a pin.

The Current Info panel settings and controls (read-only) are as follows. Which of these appears on the display depends on whether you are viewing the Source or Capture display and on the DSSC instrument’s settings.

Instrument	This drop-down list displays the instruments in the pattern Instrument statement. Only the first pin in a pin group of that instrument is shown. Other pins in the pin group appear in the ADS display in the right half of the display.
Mode	Parallel or Serial.
Shift Direction	MSB/LSB First. For serial mode only. No value displayed for parallel mode.
Bits Per Sample	Bit width or number of bits used to store samples.
Connected To	(Source) The connected status of the instrument. Connected to a list of Pins or a Pattern.

Datatype	(Capture) The data type to which to set the DSP wave for data processing during the instrument-initiated move (IIM) process.
Receive Data	(Capture) Type of data (logic or pass/fail) the instrument reports for this pin.
AutoCond	Enabled or Disabled. DSSC mode that allows you to enable a DSSC Instrument on the Condition Bus before a pattern burst.
Broadcast to Multisite	(Source) Enabled or Disabled. Allows you to send the same data to multiple sites
AutoTrig	(Capture) Enabled or Disabled. A DSSC mode that allows you to capture a DSSC signal without issuing a TRIG microcode in the pattern.
Timing Mode	Single, Single 2X, Dual, Dual 2X, Quad, Quad 2X, or By16.
Format	Data format used. Can be Binary, Offset_binary, Ones_complement, Twos_complement, Sign_magnitude, Format_unknown, or Frac_twos_complement.
StoreSTV	(Capture) Enabled or Disabled. With store_stv_enable, DSSC Capture stores values to HRAM only on vectors that have the STV bit set.

The Signal controls are:

Signal	The menu includes the names of the defined signals. If a default signal has been defined, the default signal's name appears automatically in this field. If the menu contains no signal names, no signals were previously defined in VBT.
View	Brings up the WaveScope tool and plots the sourced signal segment on the WaveScope display. This allows you to visually preview and analyze the signal that is to be sourced. For DSSC Capture, you can view only the last captured signal. For DSSC Source, you can view source signals which have been sourced. Select the signal you want to view from the Instrument list. When you are running the program on a test system, WaveScope displays the samples and scales them by the amplitude. When working offline, the WaveScope displays the ideal waveform. See About WaveScope.

Edit	Brings up the PSet and Signals Editor (PSSE) with the selected signal loaded. This control is grayed out if no signals are available. See PSet and Signals Editor in the TDE section.
Load Settings	Loads the signal's timing settings as specified on the Mixed Signal Timing sheet, as well as all of the instrument's settings (for example, voltage range, amplitude, and offset).
Trigger	(Capture) This button functions similarly to a single trigger button on an oscilloscope. You can click Trigger to make a capture. Captures made from the display are shown on WaveScope and the data is then discarded.
Start	(Source) Starts sourcing the signal associated with the specified signal according to the action command selected in the menu next to the Start button.
Stop	Stops sourcing according to the action command selected in the menu next to the Stop button. The choices are: • immediately • at end

The Pins Display panel at the bottom of the display contains information about the sample bit number, including:

- Which bits are the most and least significant
- The slot and channel to which that bit is connected
- The engine that generated it
- The pin connected to that channel

Clicking a pin name opens the Digital Channel Debug Display for that pin. For more information, see:

- [HSD1000 Digital Channel Debug Display \(HSD1000\)](#)

- [UltraPin800 Digital Channel Debug Display](#) (UltraPin800)
- [UltraPin1600 Digital Channel Debug Display](#) (UltraPin1600)

<

>

2015 Teradyne, Inc. All rights reserved.

dsdsscov.1.2

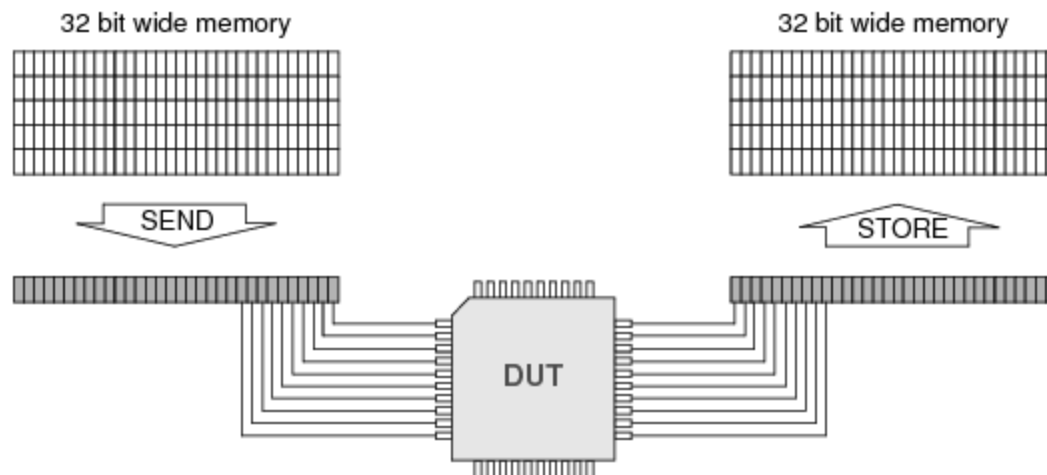


About the DSSC : Features

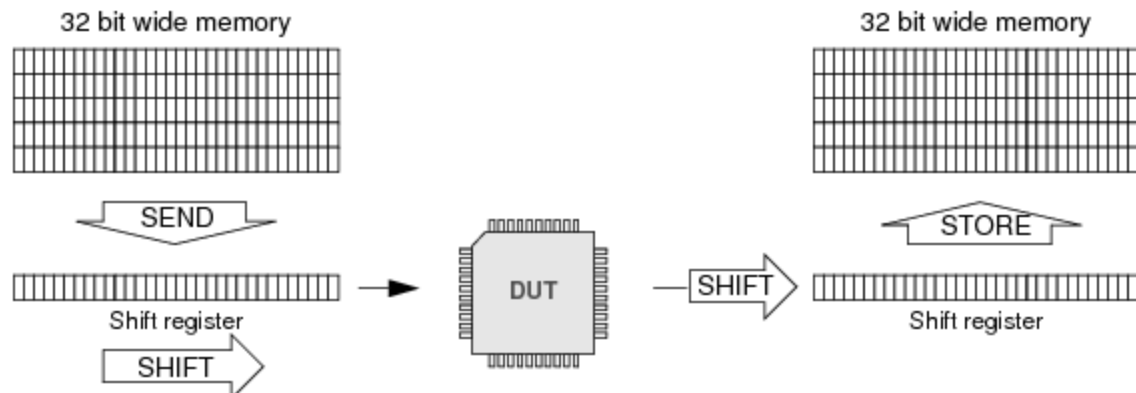
Features

The DSSC is programmed, configured, sequenced, and controlled entirely by a digital pattern. It uses digital channel electronics on a digital channel card to source data to the DUT and receive data from the DUT. Source data is defined independently of a pattern. Similarly, capture data is processed independently of a pattern.

A DSSC can source or capture data in several common data formats: parallel or serial, LSB first or MSB first, and up to 32 bits wide.



DSSC Parallel Operation



DSSC Serial Operation

Source Features

The DSSC Source provides the following features:

- [Two 32-bit Digital Source engines on each 64 channel](#) HSD1000 board in single PatGen mode (two 32-bit instruments per 64 channels in single mode and one 32-bit instrument per 64 channels in dual mode)
- [Four 32-bit Digital Source engines on each 128 channel](#) UltraPin800 board in single PatGen mode (two 32-bit instruments per 64 channels in single mode and one 32-bit instrument per 64 channels in dual mode)
- [Eight 32-bit Digital Source engines on each 256 channel](#) UltraPin1600 board in single PatGen mode, (two 32-bit instruments per 64 channels in single mode, one 32-bit instrument per 64 channels in dual 1X mode, one 16-bit instrument per 64 channels in dual 2X and quad 1X modes, and one 8-bit instrument per 64 channels in quad 2X mode)
- [2Gbit source memory](#)
- [64Megasamples maximum, for each source engine](#)
- [Single or dual modes for HSD1000 and UltraPin800](#)
- [Single, dual, or quad mode for UltraPin1600](#)
- [Serial or parallel modes](#)
 - [Serial source, 1 to 32 bits, up to 500Mbps, MSB and LSB first](#)
 - [Parallel source, 1 to 32 bit wide data up to 500Megasamples per second, 16 bit wide data up to 500Megasamples per second](#)
- [Support for 2X pin mode data rates using only one engine](#)

- 1000Mbps using dual 2X on HSD1000

- 800Mbps using dual 2X on UltraPin800

- 1600Mbps using quad 2X on UltraPin1600

- Single-ended or differential operation for source

- Ability to start signals using signal names

- When using one engine, up to 256 different signal names can be used per instrument per pattern burst

- When using multiple engines, up to 16 different signal names can be used per instrument per pattern burst

- Automatic loading of source memory

- Automatic type and format conversion

Capture Features

The DSSC Capture provides the following features:

- For HSD1000:

- Up to eight serial capture engines on each board

- Up to eight parallel capture engines on each board in dual mode

- For UltraPin800:

- Up to 16 serial capture engines on each board

- Up to 16 parallel capture engines on each board in dual mode

- For UltraPin1600

- Up to 32 serial capture engines on each board

- Up to 32 parallel capture engines on each board in dual mode

- Up to 32 parallel capture engines on each board in quad mode

- 64Megasamples maximum for each capture engine

- 1Gbit capture memory for each eight-channel group

- Single or dual modes for HSD1000 and UltraPin800

- Single, dual, or quad mode for UltraPin1600

- Serial or parallel modes

- Serial capture, 1 to 32 bits, up to 500Mbps, MSB first or LSB first

- Parallel capture, 1 to 32 bits

- Support for 2X pin mode data rates

- 1000Mbps using dual 2X on HSD1000

- 800Mbps using dual 2X on UltraPin800

- 1600Mbps using quad 2X on UltraPin1600

- Single-ended or differential operation for capture

- Background and automatic move of captured data using Instrument-Initiated Move (IIM)

- [Automatic type and format conversion](#)

- [Use of multiple time domains](#)

Typical Applications

Typical applications for the Digital Signal Source and Capture instrument include:

- [ADC and DAC testing](#)

- [Register reads and writes](#)

- [Sourcing expect data to digital channel comparators](#)

<

>

dsdsscov.1.3

<	>
-----------------	-----------------

About the DSSC : Safety Information

Safety Information

Follow general safety procedures when operating DSSC instruments. Read all system and instrument safety instructions before operating the test system. See About Operator and Safety Information in the Safety section for information.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscov.1.4

<

>

About the DSSC : DSSC Software Licensing Requirements

DSSC Software Licensing Requirements

IG-XL enforces a DSSC license if any channel on a digital board uses a DSSC engine. You must have one license for each digital instrument board that uses DSSC engines.

Note:

On UltraPin800 and UltraPin1600, a DSSC license is sold in a bundle of 128 pins, that is, the part number contains two 64 pin licenses. IG-XL enforces them as two 64 pin blocks.

Digital Board DSSC License

License Type	License Name	Enabling Points	Comments
DSSC	HSD_DSSC	1 (on) or 0 (off)	This license enables use of Digital Signal Source and Capture engines on a per PatGen basis. Each license enables 64 channels on a single board. When enforcing this license, IG-XL counts the number of instrument boards with at least one channel that requires a DSSC license. For every instrument board with at least one channel that requires a DSSC license, IG-XL requires one DSSC license for every 64 channels on the instrument board. So, even if only one channel uses DSSC on an UltraPin800 or UltraPin1600, IG-XL still requires two licenses.

IG-XL uses the same license key for all HSD1000, UltraPin800, and UltraPin1600 instrument boards when determining whether sufficient DSSC licenses are available in the test system.

In an offline simulated or current configuration file, this license appears as follows:

```
<LICENSE_ENABLERS>  
HSD_DSSC      1      4      1,0  
</LICENSE_ENABLERS>
```

For more information, see:

- [HSD1000 Software Licensing Requirements](#)
- [UltraPin800 Software Licensing Requirements](#)
- [UltraPin1600 Software Licensing Requirements](#)
- [IG-XL Licensing](#)

dsdsscprog.3.01

<	>
----------------------	----------------------

DSSC Programming

DSSC Programming

Digital source programming is based on the concept of source signals. A source signal can be a representation of an analog waveform or a set of discrete digital data. You specify signals in a test program flow using sheets and VBT. They are converted to digital samples by IG-XL based on signal properties such as amplitude and offset and DSSC settings such as instrument width and data format.

Digital capture programming is also based on the concept of capture signals. A capture signal can be a representation of an analog waveform or a set of discrete digital data. It specifies the properties of the capture DSPWave and the relevant instrument settings, such as number of samples. Instrument settings, such as instrument width and data format, also affect the conversion from samples to DSPWave. DSPWaves are the objects used for DSP processing.

The following topics are available:

- [Programming Concepts](#)
- [Programming Overview](#)
- [DSSC DataTool Programming](#)
- [DSSC Pattern Control](#)
- [Creating Source Signals](#)
- [Creating Capture Signals](#)

For more information, see:

- [DSSC \(VBT Syntax Reference\)](#)

- [DSSC Examples](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.02

<	>
-----------------	-----------------

DSSC Programming : Programming Concepts

Programming Concepts

The following sections provide conceptual information about DSSC programming:

- | | |
|---|--|
| • | Multiple Engine Digital Source Instruments |
| • | Broadcast-to-Multisite |
| • | Multiboard Instruments |
| • | Differential DSSC Channels |
| • | 2X Mode for DSSC |
| • | Storing on STV |
| • | DSSC Reset Values |

<	>
-----------------	-----------------

dsdsscprog.3.03

<

>

DSSC Programming : Programming Concepts : Multiple Engine Digital Source Instruments

Multiple Engine Digital Source Instruments

An UltraFLEX digital channel card has two 32-bit Digital Source (DigSrc) engines per 64 channels in single mode, so that:

- A single HSD1000 board supports two DigSrc engines.
- A single UltraPin800 board supports four DigSrc engines.
- A single UltraPin1600 board supports eight DigSrc engines.

Each DigSrc engine can independently source a unique signal. For example, when using an UltraPin800, you can assign one engine behind channel 0, a second engine behind channel 1, then a third behind channel 64, and a fourth engine behind channel 65. If you specify multiple DigSrc instruments with pins in the same group of 64 channels, IG-XL programs the channels in multiple-engine mode. You can then program these instruments separately, and IG-XL reconfigures the digital channels appropriately.

Multiple-engine DigSrc instruments allow you to use the same set of features available for single-engine DigSrc instruments. When using single-engine and multiple-engine DigSrc instruments, you can:

- Use the full complement of channels (from 1 to 32) per engine
- Configure each engine independently to run in parallel or serial mode

- Configure each engine independently to run in single-ended or differential mode
- Configure virtual instruments that span multiple groups of 64 channels
- Use keepalive to source multiple independent segments

The following limits apply to multiple-engine DigSrc instruments:

- You can use a maximum of 16 segments per instrument.
- You can use only single timing mode (dual and quad mode not supported).
- You can use only 1X pin data mode (2X mode not supported). Note that this is also true for single-engine DigSrc instruments.

To program a multiple-engine DigSrc instrument:

1. On a Pin Map sheet, specify separate pin groups for each instrument.
2. On a Channel Map sheet, assign pins to channels so that you are following the virtual instrument rules. See DigSrc Virtual Instrument Rules.

The Channel Map sheet below shows an HSD1000 in slot 1, an UltraPin800 in slot 2, and an UltraPin1600 in slot 3.

Channel Map

DIB ID: View Mode:

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
d0		VO	1.ch0	1.ch4
d1		VO	1.ch1	1.ch5
d2		VO	1.ch2	1.ch6
d3		VO	1.ch3	1.ch7
serial_D0		VO	1.ch16	1.ch32
a0		VO	2.ch0	2.ch64
a1		VO	2.ch1	2.ch65
a2		VO	2.ch2	2.ch66
a3		VO	2.ch3	2.ch67
serial_A0		VO	2.ch16	2.ch72
b0		VO	3.ch0	3.ch1
b1		VO	3.ch64	3.ch63
b2		VO	3.ch128	3.ch129
b3		VO	3.ch192	3.ch193
serial_B0		VO	3.ch16	3.ch72

Channel Map with Multiple Source Engines

3. In the pattern's instruments statement, create two or more instruments.

For the channel map listed above, the instruments statement is programmed as follows:

```
instruments = {
    (d3, d2, d1, d0):DigSrc 4:nonunique_sites;
    (serial_D0):DigSrc 16:serial:nonunique_sites;
    (a3, a2, a1, a0):DigSrc 4:unique_sites;
    (serial_A0):DigSrc 16:serial:unique_sites;
    (b3, b2, b1, b0):DigSrc 4:nonunique_sites;
    (serial_B0):DigSrc 16:serial:nonunique_sites;
}
```

Note that the DigSrc instrument for the UltraPin1600 (pins b0-b3) requires a virtual DigSrc instrument because it includes channels in different 64-channel banks.

DigSrc Virtual Instrument Rules

IG-XL supports DigSrc instruments that span multiple groups of 64 channels. On the 64-channel HSD1000, you can create a virtual DigSrc instrument by assigning channels from different instrument boards since each board supports 64 channels. On the 128-channel UltraPin800, you create a virtual instrument by using channels from both the lower 64 channels (channels 0-63) and the upper 64 channels (channels 64-127) on the same instrument board or by using channels from 64-channel groups across different instrument boards. On the 256-channel UltraPin1600, you create a virtual instrument by using channels from multiple banks of 64 channels (channels 0-63, 64-127, 128-191, and 192-255) on the same instrument board or by using 64-channel banks across different instrument boards.

A virtual DigSrc instrument can contain a mix of HSD1000 and UltraPin800 channels, but this mix cannot contain UltraPin1600 channels.

When you define a virtual DigSrc instrument using channels from multiple banks of 64 channels, it uses engines from each bank. In single mode, the virtual instrument uses one engine from all banks, leaving each bank with one available engine. This means that you can define two DigSrc instruments per 64 channels in single mode on the UltraPin800 and the UltraPin1600. In dual mode, the virtual instrument uses both engines from all banks, which reduces the number of available DigSrc instruments.

You can create up to 256 segments if you use one engine per 64 channels. If you use two engines per 64 channels, then you can create up to 16 segments per engine.

The table below lists the DigSrc segment memory depths:

DigSrc Segment Memory Depths

Instrument	Timing Mode	Segment Memory Depth
HSD1000/ UltraPin800	Single	64M for all segments in a 32 channel slice
	Dual 1X	128M for all segments in a 32 channel slice
	Dual 2X	256M for all segments in a 16 channel slice
UltraPin1600	Single	48M for all segments in a 32 channel slice
	Dual 1X	48M for all segments in a 32 channel slice
	Dual 2X	48M for all segments in a 16 channel slice
	Quad 1X	48M for all segments in a 16 channel slice
	Quad 2X	48M for all segments in a 8 channel slice

Note that the UltraPin1600 also has a minimum segment length in samples, which are listed in the table below. For segments under the minimum size, IG-XL automatically replicates your specified segment an integral number of times until the minimum size is met. While this is done, you may see that an increased amount of source memory is being used and may run out of memory sooner than expected.

UltraPin1600 DigSrc Minimum Segment Memory Lengths

Data Mode	Timing Mode	Minimum Segment Memory Length (Segments)
Parallel	Single	1024

	Dual 1X	2048
	Dual 2X	4096
	Quad 1X	4096
	Quad 2X	8192
Serial	Single	1024
	Dual 1X	1024
	Dual 2X	1024
	Quad 1X	1024
	Quad 2X	1024

dsdsscprog.3.04

<

>

DSSC Programming : Programming Concepts : Broadcast-to-Multisite

Broadcast-to-Multisite

Broadcast-to-Multisite allows you to send the same data to multiple sites to conserve DSSC resources.

The Broadcast-to-Multisite value is either `unique_sites` or `nonunique_sites`:

<code>unique_sites</code>	IG-XL allocates separate source engines and ADS lines for each site. If the Broadcast-to-Multisite value is unspecified, this is the default.
<code>nonunique_sites</code>	IG-XL allocates the same source engines for every site where possible. It also shares ADS lines for that instrument where possible. ADS line sharing is only possible for channels that are on the same ADS slice. Channels on different ADS slices can still be shared, but additional ADS lines must be consumed. In the worst case, ADS line usage is the same as if <code>unique_sites</code> is specified.

You can program Broadcast-to-Multisite using buttons on the PatternTool DSSC Instrument Options display.

You cannot use Broadcast-to-Multisite with:

- Capture instruments. Broadcast-to-Multisite is used with digital source instruments only.
- Different source data. If different sites require different source data, you cannot use Broadcast-to-Multisite.

You can use the following statements to read signals collections:

- [TheHdw.DSSC.Pins.Pattern.Source.Signals](#)

For a source instrument that is not using Broadcast-to-Multisite, this language statement returns the signals collection for the given pins, pattern, and active sites. In this case, there may be a different collection of signals on each site. However, for a source instrument that is using source Broadcast-to-Multisite, all sites share the same signals collection. This means that any information that is programmed to the collection when one site is active applies to all sites. For more information, see [Pins.Pattern.Source.Signals](#) in the VBT Syntax Reference.

- [TheHdw.DSSC.Pins.Pattern.Source.SitesAreUnique](#)

This read-only language statement returns `True` if sites are unique and `False` if sites are shared.

<

>

2015 Teradyne, Inc. All rights reserved.

dsdsscprog.3.05

<	>
-----------------	-----------------

DSSC Programming : Programming Concepts : Multiboard Instruments

Multiboard Instruments

Digital Source and Capture instruments can span boards. This means that you can define a single digital source or capture instrument that includes pins from multiple digital boards. You can then use that instrument as you would use any single-board DSSC instrument.

To create a multiboard DSSC instrument:

- | | |
|----|--|
| 1. | On a Pin Map sheet, create a pin group that includes pins from different digital boards. |
| 2. | In the pattern's instruments statement, define an instrument using the pins from the group on the Pin Map sheet. |

Multiboard instruments do not impose site restrictions, so you can use a multiboard DSSC instrument with the nonunique sites flag.

For Source instruments, IG-XL loads the source segment into source memory of all boards that contain pins in the multiboard instrument, regardless of how many pins the board contributes. For Capture instruments, bit combination occurs at the DSP stage. Because channels in the instrument span boards, IG-XL cannot use hardware to perform bit combination. Performing the combination at the DSP stage takes more time than performing the operation in the digital hardware.

The following channel map, pin map, and pattern excerpts demonstrate creating a multiboard Digital Source instrument.

These channel map entries define four digital channels spanning two digital boards:

Channel Map Entries for Multiboard Digital Source Instrument

Pin Name	Type	Site 0
PinA	I/O	3.ch0
PinB	I/O	3.ch1
PinC	I/O	5.ch0
PinD	I/O	5.ch1

These pin map entries define a pin group called AllPins that includes the four channels on two boards:

Pin Map Entries for Multiboard Digital Source Instrument

Group Name	Pin Name	Site 0
	PinA	I/O
	PinB	I/O
	PinC	I/O
	PinD	I/O
AllPins	PinA	
AllPins	PinB	
AllPins	PinC	
AllPins	PinD	

This ASCII pattern header section creates a multiboard Digital Source instrument based on the AllPins pin group:

```
digital_inst = hsdm;  
pinmap_workbook = "Pinmap.Txt"  
opcode_mode = dual;  
instruments = { (AllPins) : DigSrc 4:parallel; }
```

dsdsscprog.3.06

<	>
---	---

DSSC Programming : Programming Concepts : Differential DSSC Channels

Differential DSSC Channels

DSSC supports differential programming for source and capture. The same functionality and options are available for differential channels as for single-ended channels.

To use the DSSC in differential mode, you must use a Pin Map sheet to define a pin pair containing true pin and complement. For more information on programming differential digital channels, see:

- [HSD1000 Differential Operation \(HSD1000\)](#)
- [UltraPin800 Differential Operation \(UltraPin800\)](#)
- [UltraPin1600 Differential Operation \(UltraPin1600\)](#)

Once you create a differential pin group, you control the pin pair by programming the true pin only. When creating pattern vectors, you supply the true data; IG-XL determines the complement data and programs the complement pin appropriately.

For example, assuming you have created differential pin groups named diff1 through diff8, you can then use them in a pattern similar to the following:

```
opcode_mode = single;
digital_inst = hsdm;
import tset ts0;
instruments = {
    (diff1, diff2, diff3, diff4) : DigSrc 4;
    (diff5, diff6, diff7, diff8) : DigCap 4; }
vm_vector
module 1 ( $tset, diff1, diff2, diff3, diff4, diff5, diff6, diff7, diff8)
```

```
{
    > ts0  H H H H  1 1 1 1 ;
((diff5, diff6, diff7, diff8):DigCap = trig)
((diff1, diff2, diff3, diff4):DigSrc = start wave4)
    > ts0  H H H H  1 1 1 1 ;
repeat 144 > ts0  H H H H  1 1 1 1 ;
((diff5, diff6, diff7, diff8):DigCap = store)
((diff1, diff2, diff3, diff4):DigSrc = send)
repeat 32 stv > ts0 D D D D  V V V V ;
((diff1, diff2, diff3, diff4):DigSrc = stop)
.
.
}
```

You can then run the pattern from VBT as you would run a single-ended pattern.

Differential Pattern Data

In 1X mode, you can use the following pin data combinations:

- | | |
|---|--|
| • | Pin state characters without Capture extended microcodes: 0, 1, L, H, M, V, X, -, D, I, E, and C |
| • | Combinations of receive states L, H, M, V, X, E, and C with any Capture extended microcode combination |
| • | Combinations of 0, 1 and Trig |

In 2X mode, you can use the following pin data combinations:

- | | |
|---|---|
| • | Any early/late combination from 0 and 1 |
| • | Any early/late combination from L, H, and X |
| • | Any early/late combination from M and X |
| • | Any early/late combination from V and X |
| • | 0X, 1X, DD, II, EE, CC |

- Any early/late combination from L and H, with Capture extended microcode Store
- VV or XX with DigCap extended microcode Store
- VV or XX with DigCap extended microcode Trig
- VV or XX with DigCap extended microcodes Trig and Store
- VV/Store, Inst_Cond_Strobe

For more information, see [Pin State Characters and Extended Microcodes](#) in the Pattern Language section.

In differential mode, Digital Capture receives the output of the differential comparator. It does not capture complement data or common mode data.

<

>

2015 Teradyne, Inc. All rights reserved.

dsdsscprog.3.07

	
---	---

DSSC Programming : Programming Concepts : 2X Mode for DSSC

2X Mode for DSSC

DSSC supports dual 2X mode for both Source and Capture instruments. In 2X mode, the instrument provides two states per vector, permitting double the data rate. You can use 2X mode with both single-ended and differential channels. DSSC does not support single 2X mode.

To configure a DSSC instrument for 2X mode, program the appropriate HSD channels to 2X mode, including timing, pin setup, and other parameters. As the DSSC takes its channel setup from the underlying HSD board, programming the HSD channels also configures the DSSC instrument. For more information on programming HSD timing and pin setup, see:

- [Timing and Edge Programming Methods \(HSD1000\)](#)
- [Timing and Edge Programming Methods \(UltraPin800\)](#)
- [Timing and Edge Programming Methods \(UltraPin1600\)](#)

For valid pin data combinations in differential mode, see [Differential Pattern Data](#).

When using 2X mode with a DSSC Source, note the following restrictions:

- You must use dual PatGen mode and true differential mode.
- In parallel mode, words can be from 1 to 16 bits wide. In serial mode, you can use the maximum 32-bit word width.
- You can use the following combinations of ADS pin data: D, D; I, I; E, E; C, C.

When using 2X mode with a DSSC Capture, the only supported microcode and pin data combination for single-ended channels is store V, V. To use triggers and conditions, you must set up auto_trig and auto_cond. For more information on auto-trigger and auto-condition, see [DSSC Instruments Statement](#).

 Note:

Each set of eight channels on a single HSD board (for example, channels 0 to 7, channels 8 to 15) can support four DSSC capture bits in 2X mode. If a given set of channels requires more than four bits (for instance, five channels marked as DSSC 2x on the same instrument or shared between two instruments on the same board), IG-XL may return unreliable results.

<

>

2015 Teradyne, Inc. All rights reserved.

dsdsscprog.3.08



DSSC Programming : Programming Concepts : Storing on STV

Storing on STV

DSSC can store data on the store microcode or the pattern generator's STV microcode. Because IG-XL implements the store microcode as part of pin data, you can use it to specify exactly which data to store and on which pins. However, it reduces the number of pin states available for representing data. IG-XL implements the STV microcode as a control bit in the hardware, which decouples it from pin data and makes all pin states available for defining data. IG-XL also supports the use of MTO and SCAN while using the STV microcode to store data.

The STV microcode stores data in the tester's capture memory (CMEM), which can store large captures. You can also use an instrument-initiated move (IIM) to transfer the data to the tester's DSP computer in the background. Your capture is stored in a DSPWave object, and you can use standard DSP methods to analyze the information. You can also use deferred limits testing.

On HSD1000 and UltraPin800, if one of the vectors in a vector group does not fire a strobe, then data saved for that vector is a binary 1 regardless of the actual data. On UltraPin1600, however, if one of the vectors in a vector group does not fire a strobe, then the data saved for that vector is a binary 0, which supports DigCap in the edge compare mode. Vectors that do not fire strobes include vectors with drive data (0 or 1) and vectors with compare data (L, H, M, V, and X) that do not have strobe timing defined or strobes disabled.

An additional pin setup is available when storing using STV. The normal setup provides the same states that are available when you do not specify a setup for a standard HSD pin. This allows you to instruct the pattern compiler to use these states for a DSSC Capture pin. In standard mode, you can use the 0, 1, L, H, M, V, X, and - states. In differential mode, you can use the 0, 1, L, H, M, V, X, -, D, I, E, and C states. For more information, see the default setup for non-MTO pins in [Default Pin Setup](#) in the Pattern Language section.

To configure a DSSC instrument for STV, set the store_stv option of the instruments statement to store_stv_enable.

When you store data using the STV microcode, observe the following restrictions. These apply to all DSSC Capture instruments in a pattern burst, including pattern sets.

- Keep the store_stv_enable and receive_data settings the same.
- Do not use the store microcode.
- Do not use the SetCaptureConfig method to set up the pattern burst.

Related Topics

- StoreSTV in the VBT Syntax Reference for DSSC
- DSSC Instruments Statement for DSSC instruments

dsdsscprog.3.09

<

>

DSSC Programming : Programming Concepts : DSSC Reset Values

DSSC Reset Values

The following table lists the local reset values for the DSSC. Pre-job reset, post-job reset, power-up reset, and user reset all result in the same state.

Digital Signal Source and Capture Reset States	
Property	Value
Amplitude	1 LSB
Cycle count	1
DC offset	0V
Default signal name	"" (null string)
Instrument condition sample	Disabled
Phase	0 degrees
Sample rate	0Hz
Sample size	0 for DSSC Source, 1 for DSSC Capture
Samples	0
Wave definition name	"" (null string)

In addition, the .Reinitialize methods for both Source and Capture signals:

- Disable all DSSC engines

- [Set the instrument to source mode](#)

- [Set all DSSC engines to site 0](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.10

<	>
-----------------	-----------------

DSSC Programming : Programming Overview

Programming Overview

You program the DSSC primarily through entries in a digital pattern. For completeness, all of the steps in a test program are described.

The programming procedure for a complete test program requires the following steps:

- | | |
|----|---|
| 1. | Create a pin map with pins of the device under test (see Programming the Pin Map). |
| 2. | Create a channel map with the digital channels that will be used (see Programming the Channel Map). |
| 3. | Add global specifications. |
| 4. | Add AC specifications. |
| 5. | Add DC specifications. |
| 6. | Set up digital channel timing (see Programming Timing). |
| 7. | Set up digital channel pin levels (see Programming Pin Levels). |
| 8. | Create DSSC patterns (see DSSC Pattern Control). |

9.

Create a test procedure to source the digital segments and capture the digitized signal (see [Creating Source Signals](#)).
10.

Create the waveforms (segments) to be sourced (see [Creating Source Signals](#)).
11.

Instantiate a test for each time you wish to use either the Digital Source or Digital Capture, creating test instances for the DSSC as needed (see [Inserting a Test in the Flow](#)).

[VBT syntax](#) is described in the [DSSC](#) section of the [VBT Syntax Reference](#).

<

>

2015 Teradyne, Inc. All rights reserved.

dsdsscprog.3.11

<	>
-----------------	-----------------

DSSC Programming : DSSC DataTool Programming

DSSC DataTool Programming

This section describes programming DataTool sheets for use with the DSSC and includes the following topics:

- | | |
|---|---|
| • | Programming the Pin Map |
| • | Programming the Channel Map |
| • | Programming Timing |
| • | Programming Pin Levels |

For a basic programming procedure, see [Programming Overview](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.12

<

>

DSSC Programming : DSSC DataTool Programming : Programming the Pin Map

Programming the Pin Map

A pin map sheet describes the connections from the tester to the DUT. Since the DSSC connects to the DUT through digital channels, the DSSC does not have entries in the pin map. Each digital source and capture is identified by a pin and that pin in a pattern.

You select which digital channels are to be used by the DSSC and then use the Pin Map sheet to specify that the digital channels are connected to the DUT pins. These pins must be specified as type I/O. The procedure is:

1	Pin Map			
2				
3	Group Name	Pin Name	Type	Comment
65	digsraser digsrpar	d61	I/O	
66		d62	I/O	
67		d63	I/O	
68		d30	I/O	
69		d10	I/O	
70		d11		
71		d12		
72		d13		
73	digsrpar	d14		
74	digsrpar	d15		

Pin Map Sheet Showing DSSC Pins

For DSSC pins, select I/O from the following list.

I/O	Bidirectional digital pins
Input	Digital input pins
Output	Digital output pins
Analog	BBAC pins
Power	Device power supply pin
Gnd	Ground pin

Unknown

Unknown types or other types not listed

Utility

Utility pins

For more information, see:

- Programming the Pin Map (HSD1000)
- Programming the Pin Map (UltraPin800)
- Programming the Pin Map (UltraPin1600)
- Pin Map Sheet

dsdsscprog.3.13

<

>

DSSC Programming : DSSC DataTool Programming : Programming the Channel Map

Programming the Channel Map

A channel map sheet defines how tester instruments connect to DUT pins. A channel map uses the pin map information and contains the wiring information for the device interface board (DIB). You must include a channel map in your test program.

The DSSC itself does not have entries in the channel map; instead, use the digital channels that connect the DSSC to the DUT.

Channel Map					
DIB ID:		View Mode: Pogo			
Device Under Test		Tester Channel			
Pin Name	Package Pin	Type	Site 0	Comment	
d9		I/O	18.a9	2.a9	
d10		I/O	18.a10	2.a10	
d11		I/O	18.a11	2.a11	
d12		I/O	18.a12	2.a12	
d13		I/O	18.a13	2.a13	
d14		I/O	18.a14	2.a14	
d15		I/O	18.a15	2.a15	
d16		I/O	18.a16	2.a16	
d17		I/O	18.a17	2.a17	
d18		I/O	18.a18	2.a18	
d19		I/O	18.a19	2.a19	
d20		I/O	18.a20	2.a20	
d21		I/O	18.a21	2.a21	
d22		I/O	18.a22	2.a22	
d23		I/O	18.a23	2.a23	
d24		I/O	18.a24	2.a24	
d25		I/O	18.a25	2.a25	
d26		I/O	18.a26	2.a26	
d27		I/O	18.a27	2.a27	
d28		I/O	18.a28	2.a28	
d29		I/O	18.a29	2.a29	
d30		I/O	18.a30	2.a30	

Channel Map Sheet Showing Digital Channels

For more information, see:

- Programming the Channel Map (HSD1000)

- [Programming the Channel Map \(UltraPin800\)](#)
-
- [Programming the Channel Map \(UltraPin1600\)](#)
-
- [Channel Map Sheet](#)

<

>

dsdsscprog.3.14

<

>

DSSC Programming : DSSC DataTool Programming : Programming Timing

Programming Timing

You do not program DSSC timing directly. Since the DSSC connects to the DUT through digital channels, you program the digital channels for the timing that you want the DSSC to have. For more information, see:

- [Timing and Edge Programming Methods \(HSD1000\)](#)
- [Timing and Edge Programming Methods \(UltraPin800\)](#)
- [Timing and Edge Programming Methods \(UltraPin1600\)](#)

1	Time Sets						
2							
3	Timing Mode: Dual		Master Timeset Name:				
4	Time Domain:						
5							
6							
7		Cycle	Pin/Group				
8	Time Set	Period	Name	Clock Period	Setup	Edge Set	Commer
9	ts0	200. E-09	AllPins		i/o	es0	
10	ts1	200. E-09	AllPins		i/o	es0	
11							
12							
13							
BstRepair / PatSets / TSetsDual2X / ESetsDual2X / TSetsDual / ESetsDual / AC Specs / TSets / ESets / -Errors-							

Time Sets Sheet

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Edge Sets

Timing Mode: Dual
Time Domain:

Pin/Group	Edge Set	Data		Drive			
		Src	Fmt	On	Data	Return	Off
AllPins	es0	PAT	NR	200.E-09	200.E-09		

BitRepair / PatSets / TSetsDual2X / ESetsDual2X / TSetsDual / ESetsDual / AC Specs / TSets / ESets / -Errors-

Edge Sets Sheet

☒ Note:

You must use the Edge Capture option in the Compare Mode column of the Edge Sets sheet if you intend to capture data to the capture memory using the DSSC Capture or MTO.

< >		
	2015 Teradyne, Inc. All rights reserved.	

dsdsscprog.3.15

<

>

DSSC Programming : DSSC DataTool Programming : Programming Pin Levels

Programming Pin Levels

You cannot program levels for the DSSC directly. Since the DSSC connects to the DUT through the digital channels, you program the digital channels for the levels that you want the DSSC to have. For more information, see:

- Programming Pin Levels (HSD1000)
- Programming Pin Levels (UltraPin800)
- Programming Pin Levels (UltraPin1600)

Pin Levels					
	Pin/Group	Seq.	Parameter	Value	Comment
4	all		V _{ol}	1.5	
5	all		V _{oh}	1.5	
6	all		V _t	0	
7	all		V _{cl}	-1	
8	all		V _{ch}	6	
9	all		V _{outLoTyp}	1.5	
10	all		V _{outHiTyp}	1.5	
11	all		DriverMode	Largeswing-VT	
12	all		V _{ih}	6	
13	all		V _{il}	-0.5	

Pin Levels Sheet

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.16

	
---	---

DSSC Programming : DSSC Pattern Control

DSSC Pattern Control

Digital patterns control much of DSSC programming. The pattern specifies:

- Which DUT pins are to be connected to the DSSC
- How each DSSC instrument is to be used, whether source or capture
- Mode (serial or parallel)
- Number of bits per word sourced/captured
- Shift direction (MSB (default) or LSB (serial mode only))
- Numerical format of sourced or captured data

Patterns do not specify:

- Source signal properties or data
- Capture signal properties

The following topics are available:

- DSSC Instruments Statement

- [DSSC Pattern Microcodes](#)
- [DSSC Source and Capture Pin Data](#)
- [Dual Mode DSSC Patterns](#)
- [Quad Mode DSSC Patterns](#)

You must also follow certain rules when using [DSSC keepalive pattern modules](#). For more information, see [DSSC Keepalive Patterns](#) in IG-XL Programming Concepts.

dsdsscprog.3.17

<

>

DSSC Programming : DSSC Pattern Control : DSSC Instruments Statement

DSSC Instruments Statement

You can create patterns as ASCII text files of type .atp. To add DSSC resources to a .atp file, include an instruments statement in the header. In the following instruments statement, italicized fields are optional. Each field, if present, must be separated by a colon.

For Digital Source:

```
instruments = {  
    pin-item:digsrc instrument-width: bit-order: instrument-mode:  
    site-uniqueness: format: auto_cond;  
}
```

For Digital Capture:

```
instruments = {  
    pin-item:digcap instrument-width: bit-order: instrument-mode:  
    format: data-type: auto_cond: auto_trig_enable: store_stv: receive_data;  
}
```

where:

pin-item

Can be:

- A pin name or pin group name (defined in the pin map)
- Any combination of pins and pin groups, enclosed in parentheses and separated by commas, in the form: (item, item, ..., item)

 **Note:**

The order of pins in a parallel pin item is significant. The pin that is to drive or receive the MSB of the parallel DSSC word must be specified first in the parenthesized list or in the pin group definition.

instrument-type	The string digsrc indicates a source channel, and digcap represents a capture channel. Separate entries in an instruments statement can define the same I/O pin as both digsrc and digcap.
instrument-width	Integer value, can be 1 to 32. Required, no default. Specifies the width of the instrument used for source or capture. It is normally set to the number of bits per device word and is specified as a numerical string, for example, 16. For parallel instruments, instrument-width should equal the total number of pins in the pin items. For serial instruments, instrument-width should equal the number of bits per sample in the pattern. Note that if the instrument-width is not set correctly for serial instruments, it can lead to inadvertent leading or trailing zeroes to be shifted to or from the device. If format=binary or format=offset_binary, the maximum instrument width is 31 bits when using Instrument-Initiated Move (the 32nd bit is reserved as a sign bit).
bit-order	Can be lsb or msb. Is an optional field used only for serial modes to indicate the bit order of the data. The bit-order value is either msb, meaning the most significant bit is shifted in first, or lsb, meaning the least significant bit is shifted in first. If the DSSC is in a serial mode, and this field is not specified, it defaults to msb.
instrument-mode	Can be serial or parallel. Is an optional field used to indicate a serial or parallel instrument. If the field is not specified, it defaults to parallel.
format	Is an optional field used to indicate the format of the data being sourced or captured. The syntax is format=value. Valid values for the format are: format=binary

- `format=offset_binary`
- `format=ones_complement`
- `format=twos_complement`
- `format=sign_magnitude`
- `format=format_unknown`
- `format=frac_twos_complement`

If this field is not specified, it defaults to `format=binary`.

If this field is specified as `format=frac_twos_complement`, then you must program `data_type=double`.

For DSSC source, the data is automatically converted from a signal (continuous domain) to a set of discrete samples based on the value of the `format` parameter. For DSSC capture, the captured samples from the DUT are automatically converted to a double or long data type (based on the value of the `data_type` field) for DSP analysis during the instrument initiated move process.

If the data to be sourced with DSSC source is a signal which is already formatted to contain discrete samples, such as a signal created using the Excel range wave definition, then leave the `format` as `binary` or set it to `format_unknown`. If the data captured with DSSC capture does not match any of the listed data formats, then use `format_unknown` and convert the samples using a DSP procedure.

`data_type`

Can be:

- `data_type=default`
- `data_type=long`
- `data_type=double`

This parameter is relevant for digital capture only. It specifies the data type to set the DSP wave to for data processing during the instrument initiated move process. The syntax is `data_type = type`. Valid values for type are long and double. If left blank the data_type defaults to a particular data type corresponding to each data format. (See the format parameter above.) Typically, double data types are best for use with FFT based processing, and long data types are best when bit-wise operations are to be performed on the captured data in DSP.

`auto_cond`

Can be `auto_cond_disable` or `auto_cond_enable`.

`auto_cond` causes a condition bit to be generated when the capture is completed. The Digital Capture instrument must be enabled on the condition bus in a test element before the pattern starts.

Default is `auto_cond_disable`.

`auto_trig_enable`

Can be `auto_trig_disable` or `auto_trig_enable`.

`auto_trig_enable` does not use the trig microcode. Auto-trigger causes the first store to initiate a new capture segment and stores the first sample.

A DSSC Instrument in auto-trigger mode automatically generates a trig opcode on the first store command in the pattern, and does so on each first store after a capture segment completes.

When `auto_trig_enable` is set:

- If the number of stores overruns the expected sample size, a new capture segment starts, and no error is returned.
- If the number of stores is smaller than the expected sample size, no error is returned and the remaining capture segment data after that point is left as uninitialized data.

When `auto_trig_disable` is set:

- If the number of stores overruns the expected sample size, the extra captures are discarded, and no error is returned.
- If the number of stores is smaller than the expected sample size, an error is returned.

In single mode, auto-trigger allows a continuous capture flow from one capture segment to the next.

In dual mode, the first store that initiates a new capture can be placed on either of the two DUT cycles of a major vector. A capture segment cannot complete on the first half of a major cycle, while a new segment starts on the second half of the same major cycle.

For digital capture, the default is auto_trig_disable.

site-sharing

Can be unique_sites or nonunique_sites.

For unique_sites, separate source engines and ADS lines are allocated for each site. For nonunique_sites, the same source engines and ADS lines are shared across sites wherever possible.

unique_sites is the default.

This parameter is for source only and is optional.

See Broadcast-to-Multisite.

store_stv

Can be store_stv_enable or store_stv_disable. This parameter is for DSSC Capture only and is optional.

With store_stv_enable, DSSC Capture stores values to HRAM only on vectors that have the STV bit set. In dual PatGen mode, setting the STV bit on either minor vector stores both minor cycles. In quad PatGen mode (UltraPin1600 only), setting the STV bit on any of the four minor vectors stores all four minor cycles.

This mode imposes the following restrictions:

- You cannot use DSSC's store microcode when store-on-STV is enabled.
- If you enable this mode, you must do so for all instruments.

With store_stv_disable, values are stored to CMEM using DSSC's store microcode.

The default is store_stv_disable.

For more information, see Storing on STV.

receive_data

Can be receive_data=fail or receive_data=logic. This parameter is for DSSC Capture only and is optional.

With receive_data=fail, the DSSC Capture instrument stores the pass/fail result for each pin.

With receive_data=logic, the instrument stores the actual value received from the device.

The default is receive_data=logic.

For more information, see [Instruments Statement](#) in the Pattern Language section.

<		>	
		2015 Teradyne, Inc. All rights reserved.	

dsdsscprog.3.18

<

>

DSSC Programming : DSSC Pattern Control : DSSC Pattern Microcodes

DSSC Pattern Microcodes

DSSC pattern microcodes are instructions that can be placed on an individual vector of a digital pattern. For that vector, a microcode will instruct the DSSC to perform a particular action, such as to start a new signal for source or capture, source or capture the next word of data, or for serial modes, send or receive the next bit to or from the DSSC shift register.

Programming DSSC Microcodes

For each mode of operation, the DSSC has two categories of microcodes:

- Source microcodes can be used only with pins or channels that the instruments statement has specified as digsrc.
- Capture microcodes can be used only with pins or channels that the instruments statement has specified as digcap.

The format for specifying a DSSC microcode on a particular vector is either of the following:

(pin-list : instrument = microcode)

((pin-list1 : instrument , pin-list2 : instrument) = microcode)

Note that in the example above instrument must be the same, either digsrc or digcap.

where:

pin-list	Is the pin name or pin group or (comma-separated list of pins) as specified in the instruments statement.
instrument	Is either digsrc or digcap and corresponds to the instrument type for pin-list as defined in the instruments statement.

microcode Is either a DSSC source or DSSC capture microcode. See [DSSC Source Microcodes](#) and [DSSC Capture Microcodes](#).

For more information on instrument microcodes, see [Instrument Microcode](#) in the Pattern Language section.

DSSC Source Microcodes

DSSC Source Pattern Microcodes

Microcode	Description
nop	No operation.
send	Parallel mode: Send the next word to the pins. Serial mode: Send one bit of the next word to the pin. You must use a start microcode to start a new segment before using a send microcode. If you use a stop microcode and then a send microcode without the start microcode, the hardware can enter an unknown state which can be cleared only by a hardware reset.
start signal-name	Prefetch signal data from memory identified by signal-name . Interrupts current signal and resets to the beginning of signal-name . signal-name is optional. If no signal-name is specified, the instrument default signal property must be used.
starte signal-name	Prefetch signal data from memory identified by signal-name . Does not interrupt current signal. Begins signal-name when current iteration of current signal completes. starte microcode must occur at least 144 send microcodes before the end of the current segment in single mode. In dual mode , starte must occur at least 288 send microcodes before the end of the current segment. In quad mode , starte must occur at least 576 send microcodes before the end of the current segment.
stop	Immediately stop the source and clear the data pipeline .
inst_cond_strobe	Issue instrument condition if hardware condition flag is set .

DSSC Source Microcode Restrictions

Consider the following restrictions when using microcodes with digital source signals.

Start Microcode Latency

Any of the start signal-name microcodes are used to prefetch signal data from the Source Capture Memory. To accommodate this latency, you must have nops between a START signal-name and the first send of data from that signal. The number of nops is listed in the following table.

Number of Cycles Between start and First send		
Number of Cycles Between a Start and First Send		
HSD1000, UltraPin800, UltraPin1600, and UltraPin1600+ with Signals		
DSSC Mode	<=225MHz	UltraPin1600+ with Signals >225MHz
Single mode	144	180
Dual mode	288	360
Quad mode	576	760

Source Signal Wrapping

Unless specified differently, all source signals wrap around to be sourced again on completion. Sending all samples of a source signal is referred to as an iteration of the source signal. For serial instruments, the number of send microcodes per word should equal the number specified by instrument-width in the pattern.

DSSC Capture Microcodes

DSSC Capture Pattern Microcodes	
Microcode	Description
nop	No operation.
trig	Start a new capture segment.
store	Parallel: Store a parallel word to capture memory. Serial: Store a bit to capture memory. Note that you can use store in single mode and on minor vectors in dual mode, but you cannot use it on minor vectors in quad mode (UltraPin1600 only). See also Defining Capture Segment Sample Size.
inst_cond_strobe	Issue instrument condition if hardware condition flag is set.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsdsscprog.3.19

	
---	---

DSSC Programming : DSSC Pattern Control : DSSC Source and Capture Pin Data

DSSC Source and Capture Pin Data

Source Pin Data

To effectively source data from a DSSC source, place the symbolic data D in the pin data column of the pattern on each vector you wish to source data from the DSSC. The symbolic data D instructs the digital channel timing generators to get the channel data for that pin from the Alternate Data Source instead of VM/SRM on that vector.

Alternately, you can use the symbolic data E to source expect data from the ADS. Use this when you want to use the DSSC source instrument to source expect data to the digital channel's timing generator's comparator.

Capture Pin Data

For DSSC capture pins, you can use any symbolic receive data depending on whether you want a pattern to check for high or low levels on each cycle. For instance, using symbolic data V for DSSC capture pin data is a good choice to allow a pattern to detect erroneous midband logic levels during a DSSC capture with digital channel fail flags.

Midband levels are interpreted as low levels by DSSC capture. The V symbol allows accurate capture as pattern fail logic checks for these midband failures.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.20



DSSC Programming : DSSC Pattern Control : Dual Mode DSSC Patterns

Dual Mode DSSC Patterns

In dual mode, pattern vectors are grouped in pairs. The first vector and the next higher vector form a vector group. This grouping corresponds to the two DUT cycles in each test system vector.

- All labels must be on the first vector of the vector group.
- [send](#), and store microcode commands can be on any vector of the group. These commands are executed only once on the vector in which they appear.
- [start](#), [starte](#), [inst_cond_strobe](#), and [trig](#) microcode commands can only be on the last vector of the pair.

For more information, see:

- [Timing and Pin Data Modes](#) (HSD1000)
- [Timing and Pin Data Modes](#) (UltraPin800)
- [Timing and Pin Data Modes](#) (UltraPin1600)



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.21

<

>

DSSC Programming : DSSC Pattern Control : Quad Mode DSSC Patterns

Quad Mode DSSC Patterns

In quad mode, pattern vectors are grouped in a four-vector group. This grouping corresponds to the four DUT cycles in each test system vector.

- All labels must be on the first vector of the vector group.
- [send](#), and store microcode commands can only be on the last vector of the four-vector group. These commands are executed on all four vectors in the group.
- [start](#), [starte](#), [inst_cond_strobe](#), and [trig](#) microcode commands can only be on the last vector of the four-vector group.

For more information, see:

- [Timing and Pin Data Modes](#) (HSD1000)
- [Timing and Pin Data Modes](#) (UltraPin800)
- [Timing and Pin Data Modes](#) (UltraPin1600)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.22

<	>
----------------------	----------------------

DSSC Programming : Creating Source Signals

Creating Source Signals

Source signals are created dynamically during a test program run and require the following information to be created and sourced:

Pattern with DSSC Source instrument

A module must have a start microcode and the send microcode to send the signal to the digital channels. Must have a trig microcode or autotrig enabled.

The instrument declaration includes a format setting, which determines the format of the data in this instrument's source signals. The instrument declaration also includes an instrument_width setting, which determines how many bits of resolution are used in the signals. Digital channels should have the correct symbolic pin data to get this source data from the Digital Source instrument (see [DSSC Pattern Control](#)).

Wave Definition

This describes the shape of the waveform. The types of WaveDefinitions that return samples instead of a shape are:

- [VBT WaveDefType](#)
- [Excel Range WaveDefType](#)
- [.CreateWaveDefinition](#) language statement under TheExec.WaveDefinitions

[See About WaveDesigner](#)

Signal Name	This is used as a key under which all the information needed to create a signal is collected. It must match the signal-name used in the pattern with the start microcode. Alternatively, if no signal name is used in the pattern, the Default Signal property of the signals collection for this instrument must be set to the name of the signal being used.
Required Signal Properties	These properties complete the information required to make a set of samples to be used by the source instrument. Properties include: Number of Samples: The number of samples in the source signal. (Some Wave Definitions have intrinsic size, which will override the number of samples. Excel Range primitives have this property.)
Required Signal Properties with Non Zero Defaults	These properties complete the information required to make a set of samples to be used by the source instrument. They have default values which can be used for convenience. Properties include: - Number of Cycles: The number of cycles (periods) of the Wave Definition in the generated samples. It has a default value of 1. - Amplitude: The amplitude to apply to the Wave Definition. Most Wave Definition Primitives have envelopes from -1 to 1, some have 0 to 1, and the rest have unbounded envelopes based on the user programmed values (Excel Range Primitive, VBT Primitive, and CreateWaveDefinition language statement). It has a default value of 1. - Offset: The offset to apply to the Wave Definition. Offset is applied after amplitude. It can be used to convert bipolar Wave Definitions to unipolar samples. It has a default value of 0.
Optional Signal Properties	Specify these properties when needed: Sample Rate: This is used by coherence tools. It does not affect sample generation. It has a default value of 0. (WaveScope will display samples as data domain instead of time domain.)

- **Instrument Condition Sample:** The instrument condition sample number to be programmed into hardware if used. It has a default value of 0.
- **Resource ID:** If multiple instruments have signal data declared on the Mixed Signal Timing sheet, the Resource ID is used to distinguish which settings are for which instrument. It has a default value of "".
- **Phase:** Some Wave Definitions can be phase shifted. (for example, the sine primitive). It has a default value of 0.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsdsscprog.3.23

	
---	---

DSSC Programming : Creating Source Signals : Using Source Signals

Using Source Signals

Bringing all the information together to source a signal can be done in several ways. The required steps are outlined below.

- Declaring Wave Definition: Do this on a Wave Definitions sheet or through VBT using a CreateWaveDefinition language statement.
- Specifying Signal details: Do this using DSSC Signal language. The sequence is:
 - Add the signal to the collection
 - Set the required signal properties to the correct values
 - Load the signal settings into the instrument (see LoadSettings in the DSSC section of the VBT Syntax Reference).
- Run-time details:
 - Signals settings must be loaded into the instrument before or during the first run. Errors are returned for signals not loaded.
 - Signals are automatically loaded at pattern start time if needed.

- Signals are format converted before loading based on the instrument format and instrument_width settings.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdsscprog.3.24

<	>
-----------------	-----------------

DSSC Programming : Creating Capture Signals

Creating Capture Signals

Capture signals are specified dynamically during the test program run and require the following information to be captured:

Pattern with DSSC Capture instrument	Either this pattern, or a pattern linked or grouped to this pattern, must have a trig microcode and the store and/or shift microcodes to capture the signal from the digital channels. The instrument declaration includes a format setting, which determines the format of the data in this instrument capture segments. The instrument declaration also includes an instrument_width setting, which determines how many bits of resolution are preserved during conversion operations on the captured data. Digital channels should have the correct symbolic data to get this capture data to the ADB for capturing (see DSSC Pattern Control)
Required Signal Properties	These properties complete the information required to capture a set of samples using the capture instrument. Some have default values which can be used for convenience. Properties include: Number of Samples: The number of samples in the capture segment. See Defining Capture Segment Sample Size below.
Optional Signal Properties	Specify these properties when needed. Some have default values which can be used for convenience. Properties include:

- **DSPWave:** This object will contain the captured data. It must be used to gain access to the captured data. If multiple captures are performed with the same signal, the settings do not need to be reloaded, but a DSPWave is required for each set of capture data.
- **Number of Cycles:** The number of cycles (periods) of the capture waveform. This is used by coherence tools. It does not affect the captured data. It has a default value of 1.
- **Sample Rate:** This is used by coherence tools and DSP Analysis. It does not affect captured data, but will affect any time domain analysis. It has a default value of 1.
- **Resource ID:** If multiple instruments have signal data declared on the Mixed Signal Timing sheet, the Resource ID is used to distinguish which settings are for which instrument. It has a default value of "".

Defining Capture Segment Sample Size

Make sure that the pattern executes the same number of store microcodes as the sample size defined by the currently loaded capture signal. If you are using auto-trigger and intend to capture multiple sample sets, then ensure that the number of store microcodes in the pattern is equal to the sample size defined by the currently loaded capture signal multiplied by the intended number of sample sets to acquire.

If the number of store microcodes executed is smaller than the defined signal's capture size (as specified in the TheHdw.DSSC.Pins.Pattern.Capture.Signals.SampleSize VBT property), then the number of samples available in the DSPWave is equal to the actual number of store microcodes executed and not the size programmed in the capture signal's SampleSize property. The DSP subsystem does not return a capture timeout under this condition.

Note that in this case, the HSD1000 and UltraPin800 pad the short sample set with random sample data from CMEM to complete the capture so that the resulting DSPWave contains the identical number of samples defined in the VBT signal. On UltraPin1600, however, the DSPWave only contains the number of samples equivalent to the number of store microcodes executed by the pattern.

For example, on the UltraPin1600 where the VBT module sets the capture sample size signal property to 1000 and the pattern only performs 900 store microcodes, the digital hardware tells the

DSP that the end of the signal occurs at 900 samples once the pattern halts. The DSP does not time out, and the DSP procedure begins working on the 900 signal samples, even though this sample capture is 100 samples short relative to the capture size signal property.

If you are using auto-trigger and the number of microcodes is greater than the defined signal's capture size, but not an integer multiple of that value, then the capture queue contains extra captures that can affect subsequent measurements when sent to DSP. If the test program code does not account for these extra captures, then your test results can have a misalignment between a particular DSP procedure execution and the intended capture data set.

For example, in a case where the VBT module sets the capture sample size to 1000 and the pattern performs 1100 store microcodes with an auto-trigger in place, IG-XL sends an initial triggered capture of 1000 samples to DSP and then sends a second triggered capture of 100 samples. On any subsequent capture that uses the same capture signal name on the same DSSC capture pins elsewhere in the test program, you may have set up the test program to perform DSP on the latest capture, but instead the DSP procedure uses the 100 sample capture from the previous pattern burst. You can avoid this issue by adjusting the number of store microcodes in the pattern to be an exact integer multiple of the defined signal's capture size.

Note that if DigCap instrument spans channels on 64-channel blocks or runs out of bit combiners during a parallel capture (for instance, using five eight-bit DigCap instruments on a 64-channel block exhausts the four bit combiners on a DSSC board), then IG-XL returns an error when the number of samples is smaller than the expected sample size.

		
	2015 Teradyne, Inc. All rights reserved.	

dsdsscprog.3.25



DSSC Programming : Creating Capture Signals : Using Capture Signals

Using Capture Signals

Bringing all the information together to capture a signal can be done in several ways. The required steps are outlined below.

- Specifying Signal details: Do this using DSSC Signal language. The sequence is:
 - Add the signal to the collection
 - Set the required signal properties to the correct values
 - Load the signal settings into the instrument (see [LoadSettings](#) in the DSSC section of the VBT Syntax Reference).
- Run-time details
 - Signals settings must be loaded into the instrument before or during the first run. Errors are returned for signals not loaded.
 - Signals are format converted before being processed by DSP. This conversion is based on the [instrument](#) format, data_type, and instrument_width settings.

For information on performing these conversions, see [ConvertNumFormatTo](#).

Format Conversion

Format	Conversion Algorithms and Forms
Binary	Data_Type= Long <default for instrument_width < 31> Algorithm <pre> { signbit = 1U << (instrument_width-1); magbits = signbit-1; allbits = signbit magbits; for (i=0; i<n; i++) { vect_out[i] = vect_in[i]&allbits; } }</pre>
Binary	Data_Type= Double <default for instrument_width = 31> Algorithm <pre> { signbit = 1U << (instrument_width-1); magbits = signbit-1; allbits = signbit magbits; for (i=0; i<n; i++) { vect_out[i] = (float)(unsigned)(vect_in[i]&allbits); } }</pre>
Offset Binary	Data_Type= Long <default> Algorithm <pre> { signbit = 1U << (instrument_width-1); magbits = signbit-1; allbits = signbit magbits; for (i=0; i<n; i++) { vect_out[i] = (vect_in[i]&allbits) - signbit; } }</pre>
Offset Binary	Data_Type= Double

Algorithm

```

{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

    for (i=0; i<n; i++) {
        vect_out[i] = (float)(signed)((vect_in[i]&allbits) -
            signbit);
    }
}

```

Ones
Complement

Data_Type= Long <default>

Algorithm

```

{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

    for (i=0; i<n; i++) {
        vect_out[i] = vect_in[i]&signbit ? (vect_in[i] | ~magbits)+1 :
            vect_in[i]&allbits;
    }
}

```

Ones
Complement

Data_Type= Double

Algorithm

```

{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

    for (i=0; i<n; i++) {
        vect_out[i] = (float)(signed)(vect_in[i]&signbit ? (vect_in[i] | ~magbits)+1 :
            vect_in[i]&allbits);
    }
}

```

Sign
Magnitude

Data_Type= Long <default>

Algorithm

```

{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

    for (i=0; i<n; i++) {
        vect_out[i] = vect_in[i]&signbit ? -(int)(vect_in[i]&magbits) :
        vect_in[i]&allbits;
    }
}

```

Sign
Magnitude

```

Data_Type= Double
Algorithm
{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

    for (i=0; i<n; i++) {
        vect_out[i] = (float)(signed)(vect_in[i] & signbit? -(vect_in[i] & magbits) :
        vect_in[i] & allbits);
    }
}

```

Fractional Twos
Complement

```

Data_Type= Long
NOT SUPPORTED

```

Fractional Twos
Complement

```

Data_Type= Double <default>
Algorithm
{
    shift = 32 - num_bits;

    for (i=0; i<n; i++) {
        vect_out[i] = ((float)(vect_in[i] << shift))/(float)(unsigned int)0x80000000;
    }
}

```

Twos
Complement

```

Data_Type= Long <default>
Algorithm
{
    signbit = 1U << (instrument_width-1);

```

```

magbits = signbit-1;
allbits = signbit | magbits;
allbitscomp = ~ allbits;
for (i=0; i<n; i++) {
    vect_out[i] = vect_in[i]&signbit ? vect_in[i]|allbitscomp : vect_in[i]&allbits;
}
}

```

Twos
Complement

```

Data_Type= Double
Algorithm
{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;
    allbitscomp = ~ allbits;
    for (i=0; i<n; i++) {
        vect_out[i] = (float)(signed)(vect_in[i]&signbit ? vect_in[i]|allbitscomp :
        vect_in[i]&allbits);
    }
}

```

Format
Unknown

```

Data_Type= Long <default>
Algorithm
{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;
    allbitscomp = ~ allbits;
    for (i=0; i<n; i++) {
        vect_out[i] = vect_in[i]&signbit ? vect_in[i]|allbitscomp : vect_in[i]&allbits;
    }
}

```

Format
Unknown

```

Data_Type= Double
Algorithm
{
    signbit = 1U << (instrument_width-1);
    magbits = signbit-1;
    allbits = signbit | magbits;

```

```
for (i=0; i<n; i++) {  
    vect_out[i] = (float)(unsigned)(vect_in[i]&allbits);  
}  
}
```

<

>

2015 Teradyne, Inc. All rights reserved.

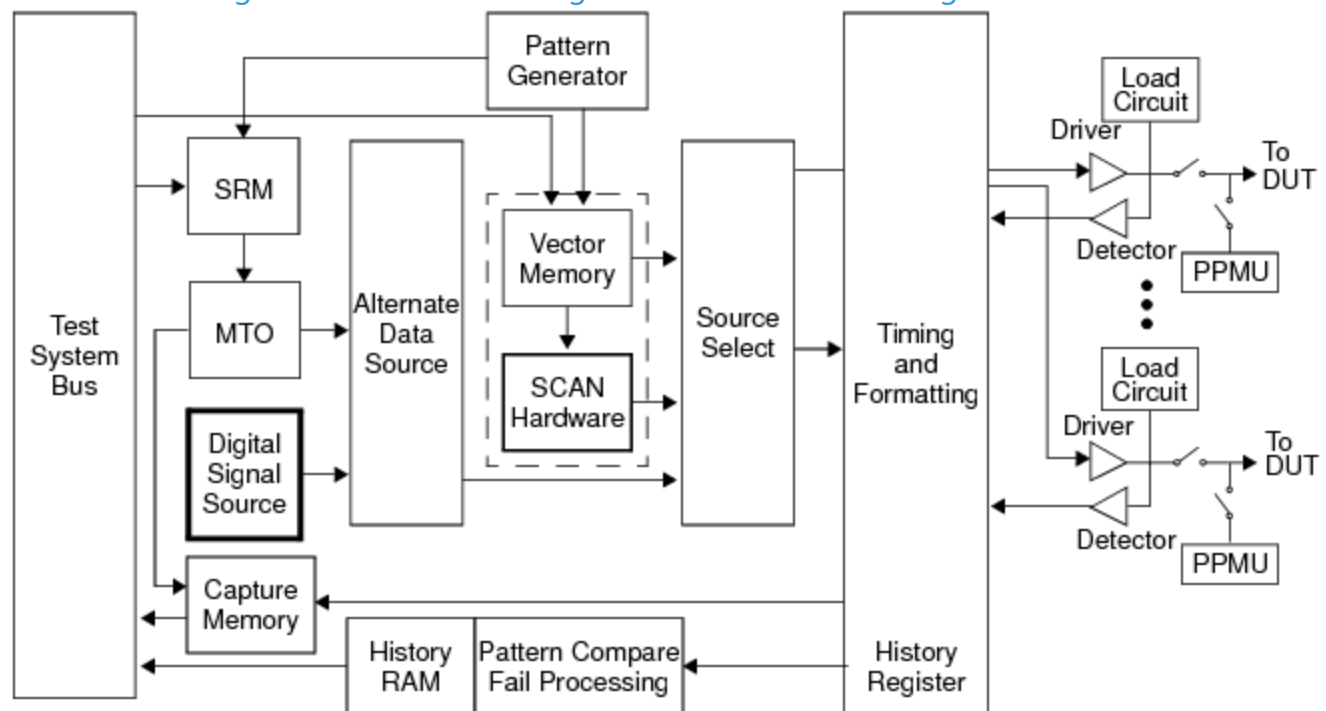
dsdssctoo.2.1



DSSC Theory of Operation

DSSC Theory of Operation

The DSSC is implemented as a part of the MTO/DSSC digital subsystem ASICs and FPGAs on the HSD1000, UltraPin800, and UltraPin1600 digital instrument boards. The following figure shows how the DSSC is integrated with the other digital instruments on the digital board.



Digital Board Functional Block Diagram

The following topics are available:

- [Digital Source](#)
- [Digital Capture](#)

This section contains information about the internal design of the DSSC that is needed to successfully lay out a DIB. The internal design of the DSSC imposes restrictions that influence channel layout on a DIB. For more information, see the following sections in the DIB Design Guidelines:

- [HSD1000 DSSC Channel Selection Guidelines](#)
- [UltraPin800 DSSC Channel Selection Guidelines](#)
- [UltraPin1600 DSSC Channel Selection Guidelines](#)

For information on DSSC performance specifications and standards, see:

- [HSD1000 Specifications](#)
- [UltraPin800 Specifications](#)
- [UltraPin1600 Specifications](#)

For more information on the digital instruments, see:

- [About the HSD1000](#)
- [About the UltraPin800](#)
- [About the UltraPin1600](#)
- [About the DSSC](#)
- [About the MTO](#)

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsdssctoo.2.2

<	>
-----------------	-----------------

DSSC Theory of Operation : Digital Source

Digital Source

This section contains information about the internal design of the Digital Source that is needed to successfully lay out a DIB. The internal design of the DSSC imposes restrictions that influence the channel layout on a DIB.

Signals

Digital source programming is based on the concept of source signals. Source signals are representations of any analog waveform or a set of discrete digital data. Signals are specified during the test program flow using a mixture of sheets, elements, and VBT. The signals are converted to digital samples by IG-XL based on signal properties such as amplitude and offset and DSSC settings such as instrument width and data format.

Digital Source Engines

An UltraFLEX digital channel card has two 32-bit Digital Source hardware engines per 64 channels in single mode, which gives you a set number of Digital Source instruments per board:

- | | |
|---|--|
| • | Two Digital Source instruments on the HSD1000 |
| • | Four Digital Source instruments on the UltraPin800 |
| • | Eight Digital Source instruments for UltraPin1600 |

Note that in dual mode, it takes two hardware engines to make one user engine, or "instrument." In quad mode, it takes four hardware engines to make one instrument.

When a pattern runs, each hardware engine can:

- | | |
|---|-------------------------|
| • | Be programmed as source |
|---|-------------------------|

- Source parallel data in these data widths:

- 1 to 32 bits in single or dual 1X mode

- 1 to 16 bits in dual 2X and quad 1X modes

- 1 to 8 bits in quad 2X mode

- Source serial data in these word lengths:

- 1 to 32 bits for single and dual 1X mode

- 2 to 32 bits, in multiples of 2, for dual 2X mode

- 4 to 32-bits, in multiples of 4, for quad 1X mode

- 8 to 32-bits, in multiples of 4, for quad 2X mode

- Source serial data LSB or MSB first

- Operate at the following data rates, depending on the digital instrument:

- HSD1000 and UltraPin800: Up to 250Mbps in single mode and up to 500Mbps in dual mode

- UltraPin1600: Up to 275Mbps in single mode, up to 550Mbps in dual mode, and up to 1100Mbps in quad mode

DSSC engines are assigned to DSSC instruments on a pattern-by-pattern basis.

For information on restrictions using multiple Digital Source engines, see [Multiple Engine Digital Source Instruments](#).

Alternate Data Source

Any digital channel can receive channel source data or channel expect data from one of two sources: memory (VM or SRM) or the Alternate Data Source (ADS). You control the data source

using channel data in a pattern; pin states D and I instruct the channel's Source Select logic to retrieve data from the ADS. The channel data also controls what receive data is sent over the ADS (see [Parallel Source](#)); pin states E and C instruct the channel's Source Select to read expect data from the ADS.

This section includes the following topics:

- [Sourcing Data](#)
- [PatGen Modes for Digital Source](#)

For more information, see [Creating Source Signals](#).

dsdssctoo.2.3



DSSC Theory of Operation : Digital Source : Sourcing Data

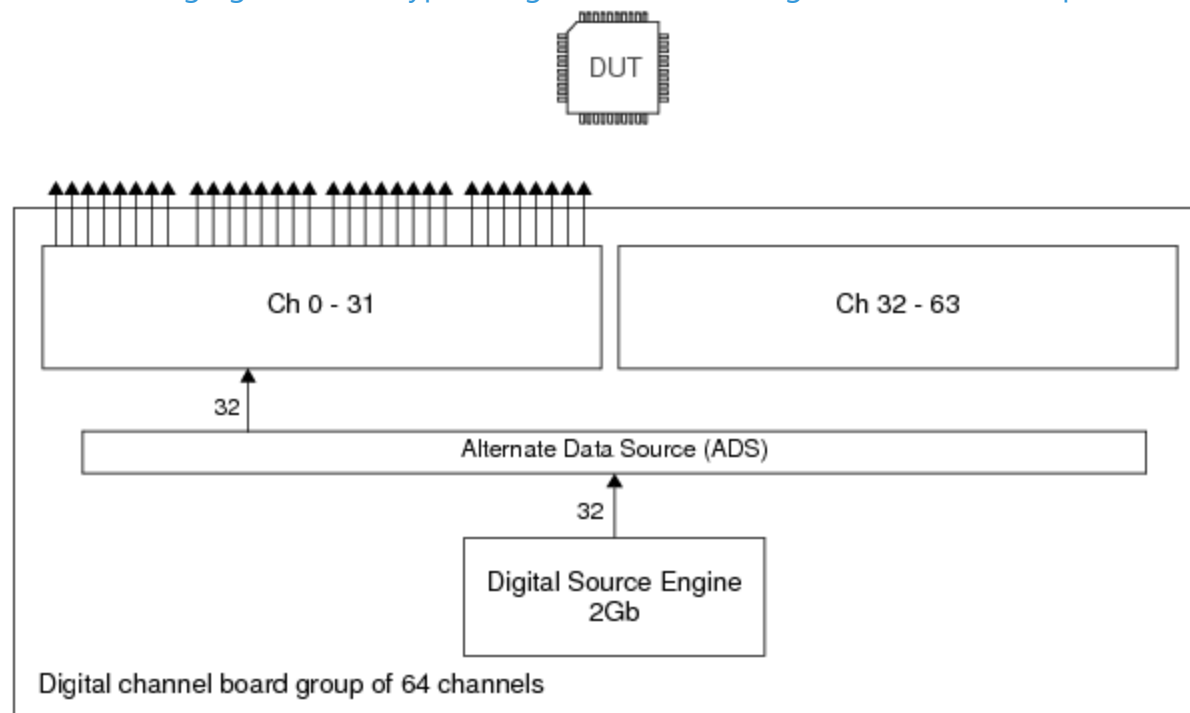
Sourcing Data

A digital source creates high speed digital waveforms by sending digital waveform data stored in digital source memory to the digital channels. Digital waveform data (source data segments) are stored in Source Segment memory.

Each Digital Source instrument can send data to digital channels in either serial or parallel mode. If multiple engines are used for parallel, the bits are stored in each engine's Source Segment memory.

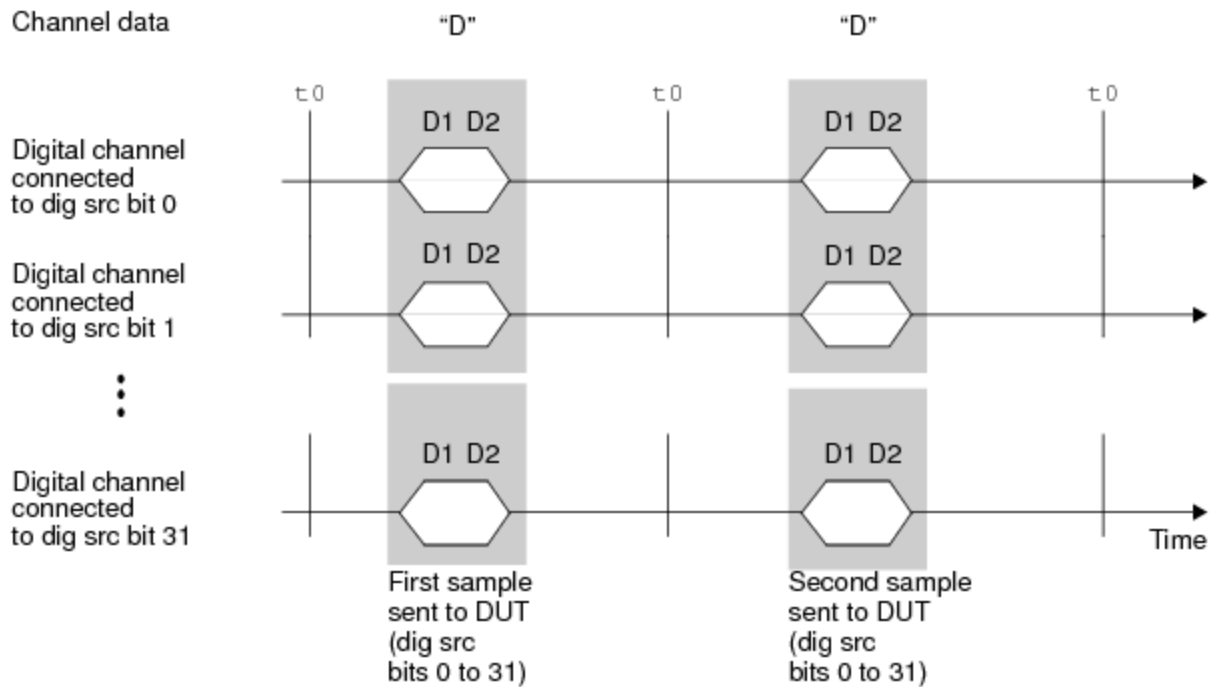
Parallel Source

In parallel mode, the DSSC can send up to 32 bits of data in each cycle to each of the digital channels connected to the DSSC. Engines can broadcast to multiple channels for multisite testing. The following figures show typical digital channel timing and data flow for parallel source.



Parallel Source

Channel data



Parallel Source Timing

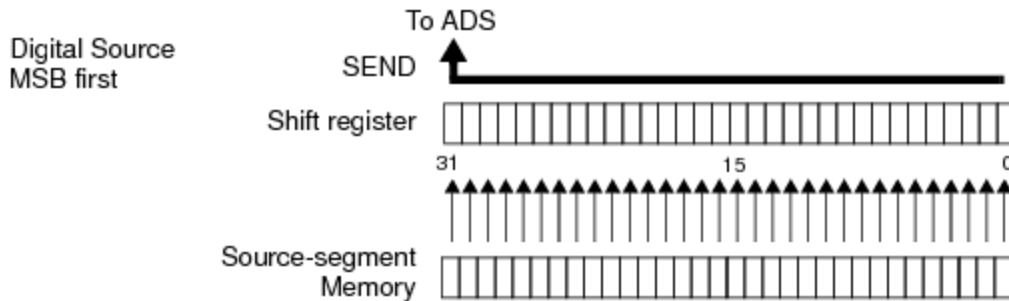
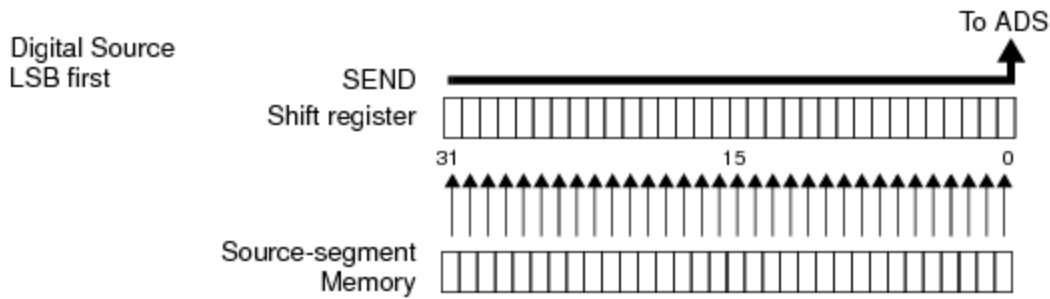
Serial Source

The DSSC can also source data in serial mode.

When data is sourced in serial mode, up to 32 bits in the Source Segment memory are loaded into the 32-bit shift register. Bit 0 of the Source Segment memory is loaded into bit 0 of the shift register and so on, up to bit 31. Once data is loaded to the shift register, the shift register under control of a DSSC engine sends data to the digital channel or channels.

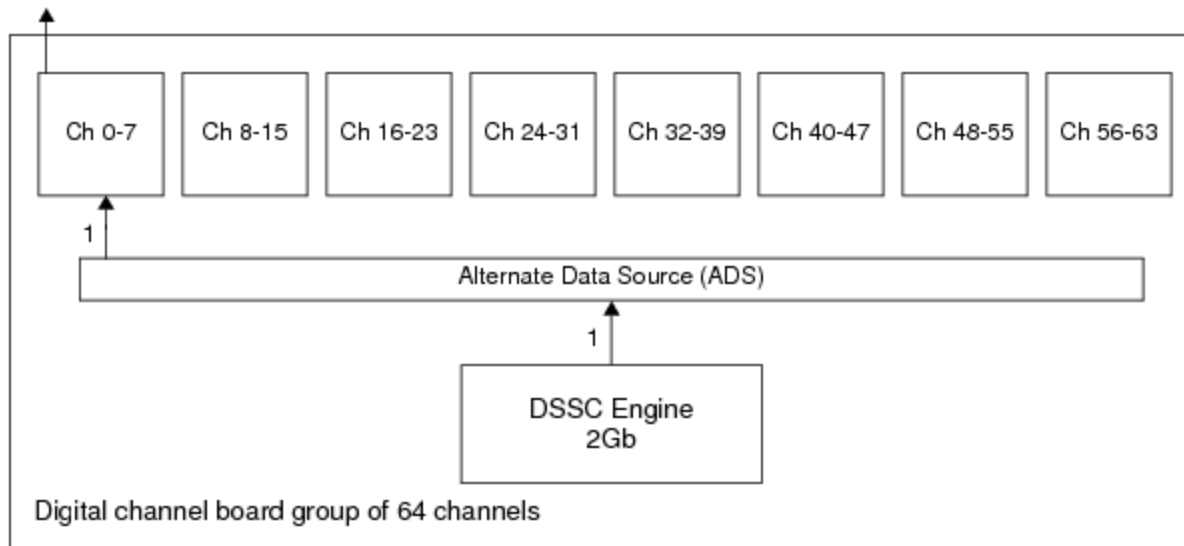
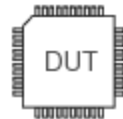
The DSSC can send data to the digital channels LSB first or MSB first.

- In **LSB first mode**, a SEND microcode command causes the DSSC to source bit 0 to the digital channel. The next SEND microcode command shifts all the remaining bits in the shift register to the right and sources bit 0 to the digital channel.
- In **MSB first mode**, a SEND microcode command causes the DSSC to source bit 31 to the digital channel. The next SEND microcode command shifts all the remaining bits in the shift register to the left and sources bit 31 to the digital channel.



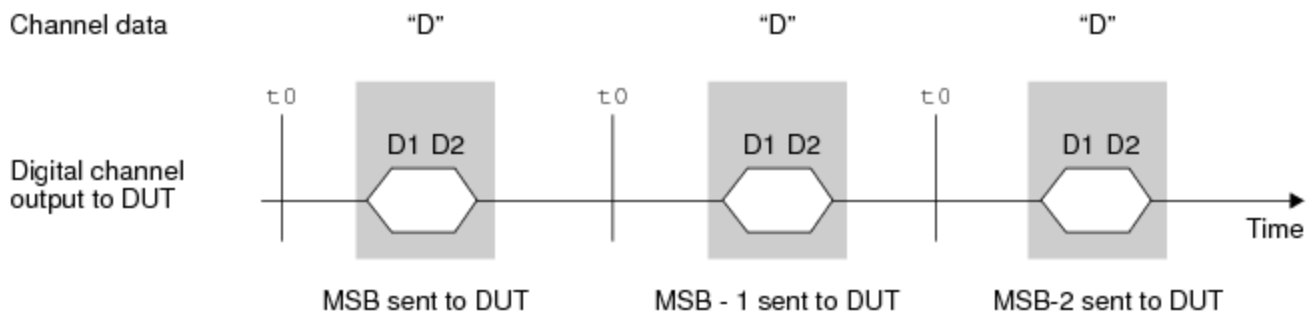
Serial Source LSB and MSB First

The following figures show digital channel timing and data flow for serial source.



Serial Source

Channel data



Serial Source Timing

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsdssctoo.2.4

<

>

DSSC Theory of Operation : [Digital Source](#) : PatGen Modes for Digital Source

PatGen Modes for Digital Source

The DSSC can operate in these Source PatGen modes:

- Single mode has a maximum data rate of 250Mbps on the HSD1000 and UltraPin800 and 275Mbps on the UltraPin1600. One vector per PatGen cycle is executed, with each vector having its own PatGen microcode.
- Dual mode has a maximum speed of 500Mbps on the HSD1000 and UltraPin800 and 550Mbps on the UltraPin1600. In dual mode, pattern vectors are grouped in pairs, where two minor vectors make up a major vector. The DigSrc data microcode (send) can be used in one, both, or neither of the vectors in a major vector. Unlike a PatGen microcode which operates on both vectors in the major vector, this DigSrc microcode operates only on the vector in which they occur. Nondata microcodes (start, stop, and inst_cond_strobe) can only occur on the last minor vector of each vector pair. See [Dual Mode DSSC Patterns](#).
- Quad mode has a maximum speed of 1600Mbps and is only available on the UltraPin1600. In quad mode, pattern vectors are grouped in a four-vector group, where four minor vectors make up a major vector. The DigSrc data microcode (send) can only be used on the last minor vector of a major vector. Unlike single and dual modes, this DigSrc data microcode operates on all vectors in a major vector. Nondata microcodes (start, stop, and inst_cond_strobe) can only occur on the last minor vector of each major vector. See [Quad Mode DSSC Patterns](#).

For more information, see [DSSC Source Microcodes](#).
The table summarizes Digital Source modes and speeds. Note that Quad and Quad 2X modes are available only on UltraPin1600.

Digital Source Modes

Data Mode	PatGen Mode	Available DigSrc Instruments	Data Width (bits per sample)	Maximum Data Rate		
				HSD1000	UltraPin800	UltraPin1600
Parallel	Single	Two 32-bit DigSrc instruments per 64 channels	1 to 32	250MT/s	250MT/s	275MT/s
	Dual 1X	One 32-bit DigSrc instrument per 64 channels	1 to 32	500MT/s	500MT/s	550MT/s
	Dual 2X	One 16-bit DigSrc instrument per 64 channels	1 to 16	1000MT/s	800MT/s	1100MT/s
	Quad 1X	One 16-bit DigSrc instrument per 64 channels	1 to 16	N/A	N/A	1100MT/s
	Quad 2X	One 8-bit DigSrc instrument per 64 channels	1 to 8	N/A	N/A	1600MT/s
Serial	Single	Two 32-bit DigSrc instruments per 64 channels	1 to 32	250Mbps	250Mbps	275Mbps
	Dual 1X	One 32-bit DigSrc instrument per 64 channels	1 to 32	500Mbps	500Mbps	550Mbps

Dual 2X	One 16-bit DigSrc instrument per 64 channels	2 to 32, in multiples of 2	1000Mbps	800Mbps	1100Mbps
Quad 1X	One 16-bit DigSrc instrument per 64 channels	4 to 32, in multiples of 4	N/A	N/A	1100Mbps
Quad 2X	One 8-bit DigSrc instrument per 64 channels	8 to 32, in multiples of 8	N/A	N/A	1600Mbps

dsdssctoo.2.5

<	>
-----------------	-----------------

DSSC Theory of Operation : Digital Capture

Digital Capture

This section contains information about the internal design of the Digital Capture that is needed to successfully lay out a DIB. The internal design of the DSSC imposes restrictions that influence the channel layout on a DIB.

Signals

Digital capture programming is based on the concept of capture signals. Capture signals can be representations of analog waveforms or a set of discrete digital data. A capture signal specifies the properties of the capture DSP Wave and the relevant instrument settings, such as number of samples. Instrument settings, such as instrument width and data format, also affect the conversion from samples to DSPWave. DSPWaves are the objects used for DSP processing.

Digital Capture Engines

An UltraFLEX digital channel card has eight 8-bit Digital Capture engines per 64 channels (eight engines for HSD1000, 16 engines for UltraPin800, and 32 engines for UltraPin1600). When a pattern runs, each engine can:

- | | |
|---|--------------------------|
| • | Be programmed as capture |
|---|--------------------------|
- | | |
|---|--|
| • | Capture data 1 to 32 bits wide in parallel, single, dual, or quad 1X mode (16 bits wide in quad 2X mode) |
|---|--|
- | | |
|---|--|
| • | Capture 1 to 32 bit word lengths in serial, single, dual, or quad mode |
|---|--|
- | | |
|---|--------------------------------------|
| • | Capture serial data LSB or MSB first |
|---|--------------------------------------|

- [On the HSD1000 and UltraPin800, operate at speeds up to 250Mbps in single mode or up to 500Mbps in dual mode](#)

- [On UltraPin1600, operate at speeds up to 275Mbps in single mode, up to 550Mbps in dual mode, and up to 1100Mbps in quad mode](#)

DSSC engines are assigned to DSSC instruments on a pattern-by-pattern basis.
This section includes the following topics:

- [Capturing Data](#)

- [PatGen Modes for Digital Capture](#)

For more information, see [Creating Capture Signals](#).

<

>

2015 Teradyne, Inc. All rights reserved.

dsdssctoo.2.6

	
---	---

DSSC Theory of Operation : [Digital Capture](#) : Capturing Data

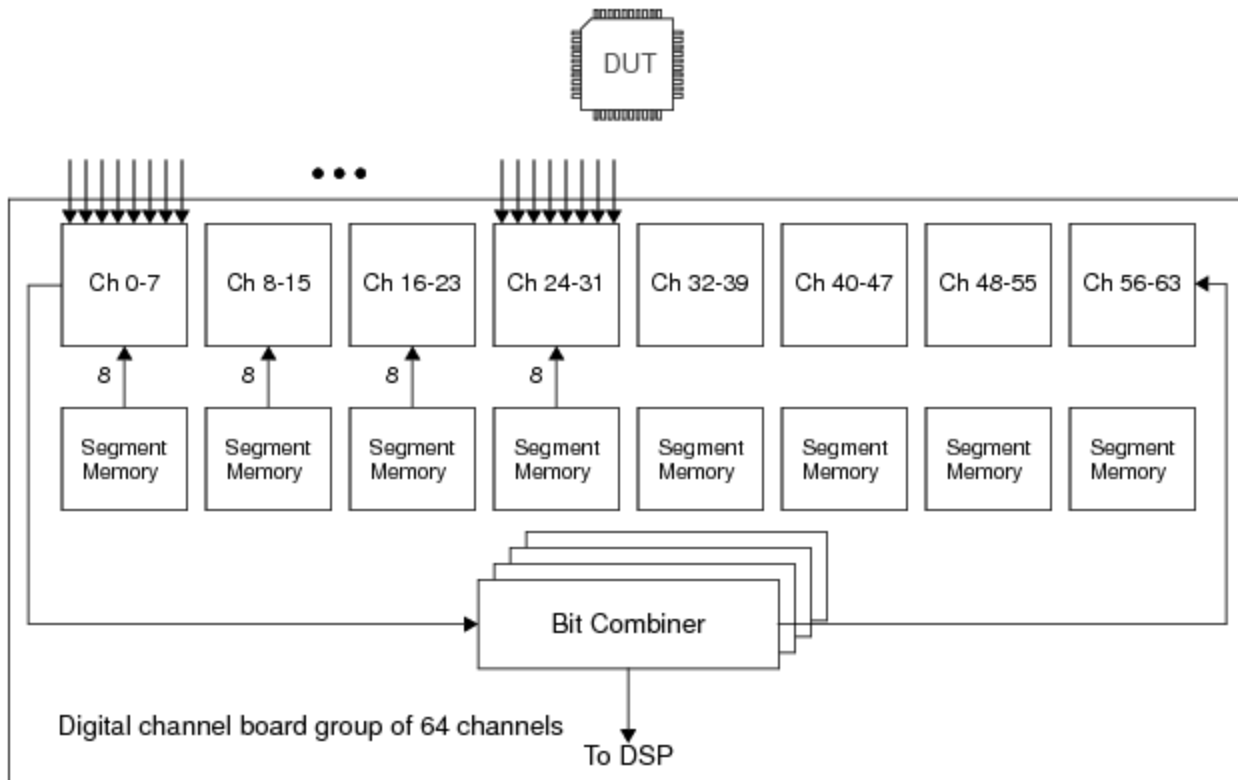
Capturing Data

Each Digital Capture instrument can receive data from the digital channels in either serial or parallel mode.

Parallel Capture

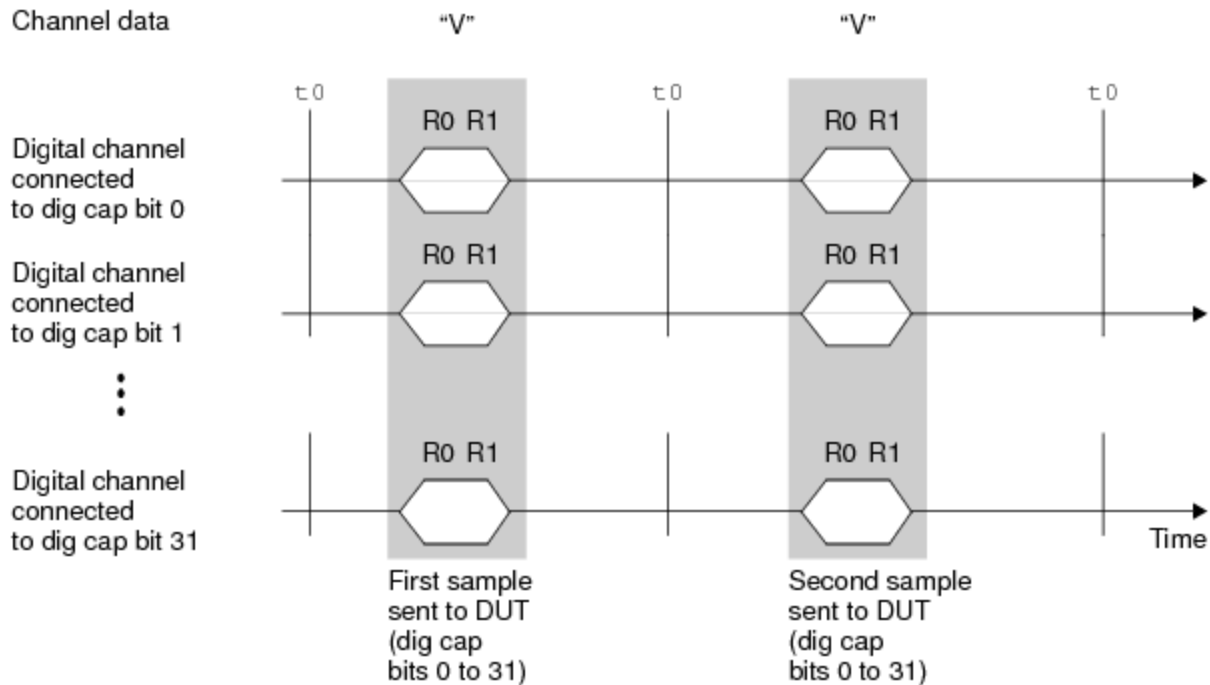
In parallel mode, the DSSC can capture up to 32 bits of data in each cycle from each of the digital channels connected to the DSSC. On HSD1000 and UltraPin800, the data can be received from the digital channels in single mode (up to 250Mbps) or dual mode (up to 500Mbps). On UltraPin1600, the data can be received from the digital channels in single mode (up to 275Mbps), dual mode (up to 550Mbps), or quad mode (up to 1100Mbps).

Once data is captured, the data is sent to the bit combiner where the data is recombined. Each board has four bit combiners.



Parallel Capture

Channel data



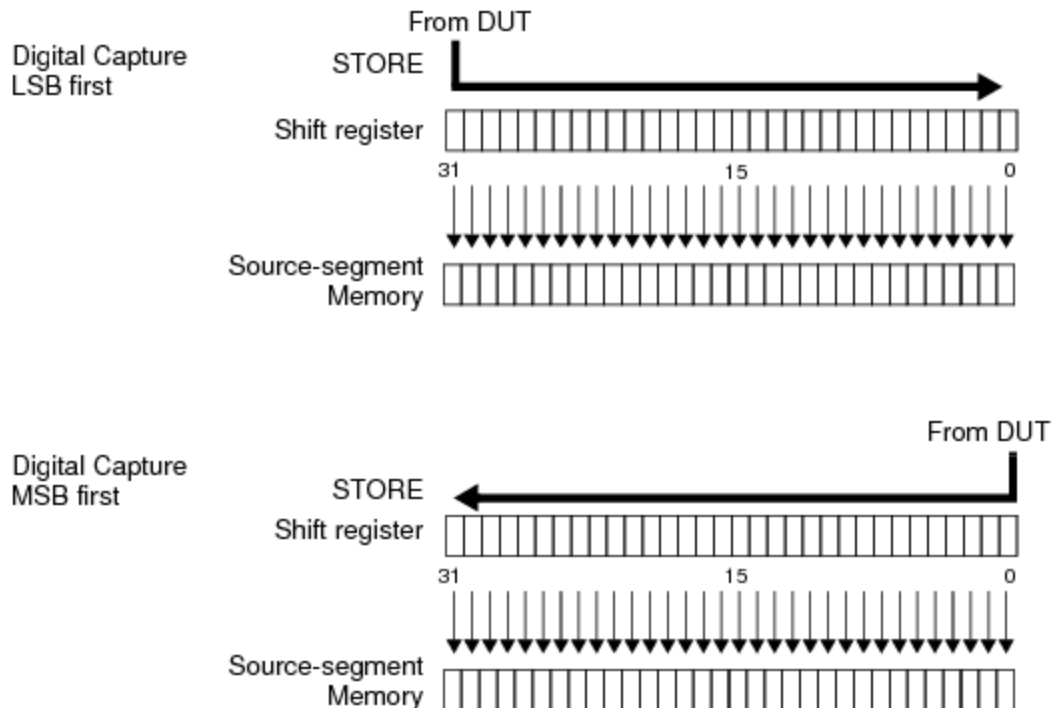
Parallel Capture Timing

Serial Capture

Serial mode captures data words LSB first or MSB first.

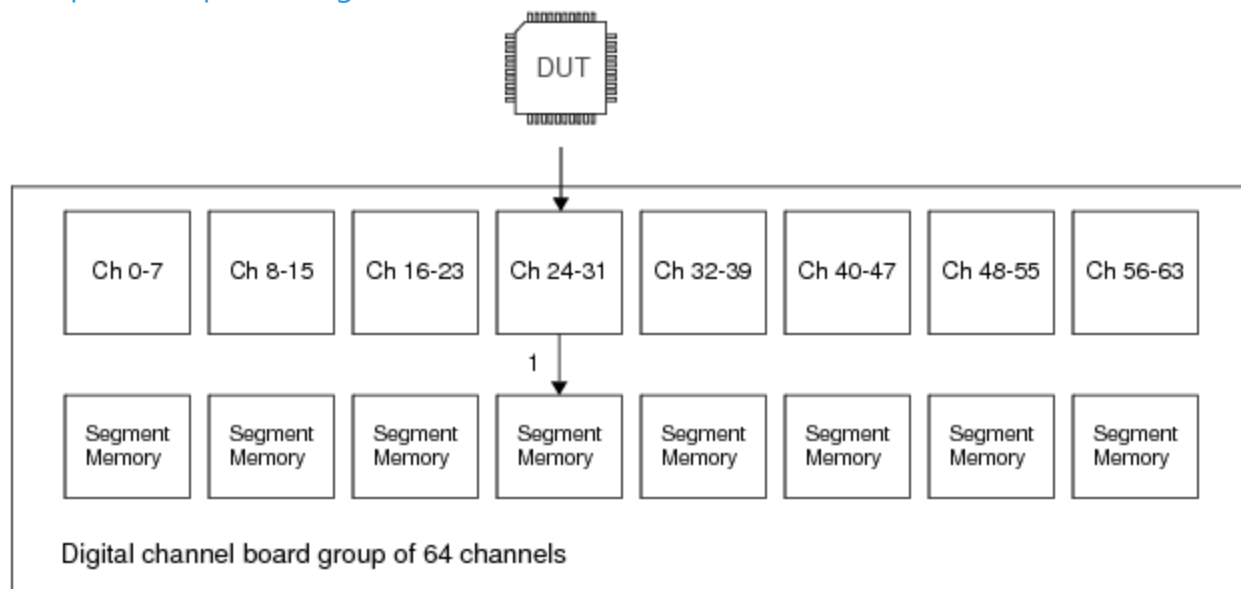
- In LSB first mode, the DSSC captures bit 0 from the DUT, then bit 1, and so on up to bit 31. A STORE microcode command captures the first bit and puts it in the shift register. The next STORE microcode command captures the next bit and moves the previous bits right.

- In MSB first mode, the DSSC engine captures bit 31 of the shift register, then bit 30, and so on down to bit 0. A STORE microcode command captures the first bit and puts it in the shift register. The next STORE microcode command captures the next bit and moves the previous bits left.



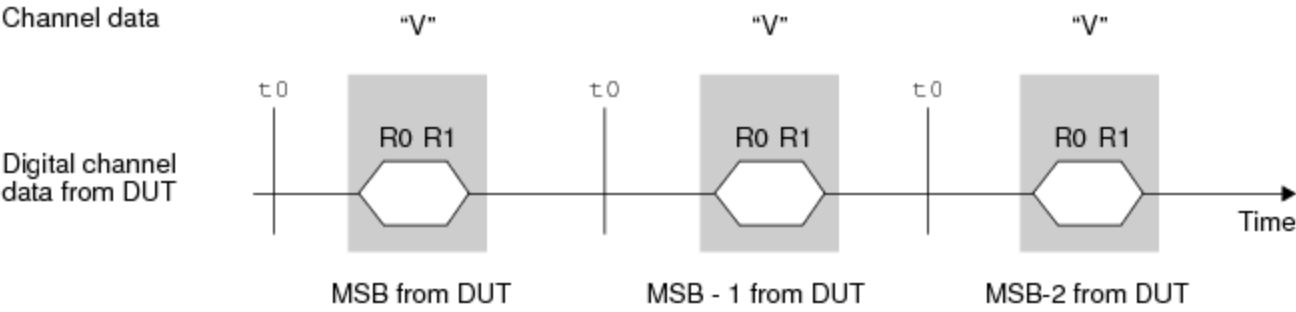
Serial Capture LSB and MSB First

Serial mode does not use the bit combiner. The serial data is sent directly to the DSP or test computer for processing.



Serial Capture

Channel data



Serial Capture Timing

<

>

2015 Teradyne, Inc. All rights reserved.

dsdssctoo.2.7

<

>

DSSC Theory of Operation : Digital Capture : PatGen Modes for Digital Capture

PatGen Modes for Digital Capture

The DSSC can operate in these Capture PatGen modes:

- Single mode has a maximum speed of 250Mbps on HSD1000 and UltraPin800 and 275Mbps on UltraPin1600. One vector per PatGen cycle is executed, with each vector having its own PatGen microcode.
- Dual mode has a maximum speed of 500Mbps on HSD1000 and UltraPin800 and 550Mbps on UltraPin1600. In dual mode, pattern vectors are grouped in pairs, where two minor vectors make up a major vector. The DigCap data microcode (store) can be used in one, both, or neither of the vectors in a major vector. Unlike a PatGen microcode which operates on both vectors in the major vector, this microcode operates only on the vector in which they occur. Nondata microcodes (trig, and inst_cond_strobe) can only occur on the last minor vector of each vector pair. See [Dual Mode DSSC Patterns](#).
- Quad mode has a maximum speed of 1600Mbps and is only available on UltraPin1600. In quad mode, pattern vectors are grouped in a four-vector group, where four minor vectors make up a major vector. The DigCap data microcode (store) can only be used on the last minor vector of a major vector. Unlike single and dual modes, this DigCap data microcode operates on all vectors in a major vector. Nondata microcodes (trig and inst_cond_strobe) can only occur on the last minor vector of each major vector. See [Quad Mode DSSC Patterns](#).

For more information, see [DSSC Capture Microcodes](#).
The table summarizes Digital Capture modes and speeds. In each of these modes, one DigCap instrument is available per eight channels, which means that in a 64-channel block, you have eight

available DigCap instruments. Note that Quad and Quad 2X modes are available only on UltraPin1600.

Digital Capture Modes						
Data Mode	PatGen Mode	Number of Channels Per Engine That Can Capture Data	Sample Width (bits per sample)	Maximum Data Rate		
				HSD1000	UltraPin800	UltraPin1600
Parallel	Single	8	1 to 32	250MT/s	250MT/s	275MT/s
	Dual 1X	8	1 to 32	500MT/s	500MT/s	550MT/s
	Dual 2X ¹	4	1 to 32	1000MT/s	800MT/s	1100MT/s
	Quad 1X	4	1 to 32	N/A	N/A	1100MT/s
	Quad 2X ²	2	1 to 16	N/A	N/A	1600MT/s
Serial	Single	1	1 to 32	250Mbps	250Mbps	275Mbps
	Dual 1X	1	1 to 32	500Mbps	500Mbps	550Mbps
	Dual 2X	1	1 to 32	1000Mbps	800Mbps	1100Mbps
	Quad 1X	1	1 to 32	N/A	N/A	1100Mbps
	Quad 2X	1	1 to 32	N/A	N/A	1600Mbps

¹ In Dual 2X, you can use four differential pairs in the same engine, using all eight channels.

² In Quad 2X, you can use two differential pairs, using four total channels.

dsscexamples.5.01

<

>

DSSC Examples

DSSC Examples

This section contains program examples for the Digital Signal Source and Capture (DSSC). The workbook examples show all IG-XL data, VBT code, and patterns necessary for a complete running program.

The following topics are available:

- DSSC Dual 2X Serial Source Test
- DSSC Dual 2X Serial Capture Test
- Using Default Signals with DSSC

Related Topics

- DSSC (VBT Syntax Reference)
- DSSC Programming

<

>

dsscexamples.5.02

<	>
-----------------	-----------------

DSSC Examples : DSSC Dual 2X Serial Source Test

DSSC Dual 2X Serial Source Test

The DSSC dual 2X serial source test sources a signal at dual 2X speeds using VBT to start the pattern. This example creates a sine wave in VBT and sources that through the DSSC.

The HSD1000 board is installed in slot 6.

This workbook example includes the following program components:

- | | |
|---|--------------------------------|
| • | DataTool Sheets |
| • | HSD Pattern with TSet and Pins |
| • | VBT Code |
| • | DSP Code |

<	>
-----------------	-----------------

dsscexamples.5.03

<	>
-----------------	-----------------

DSSC Examples : [DSSC Dual 2X Serial Source Test](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|---|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Time Sets (Basic) Sheet |
| • | Pin Levels Sheet |
| • | Test Instances Sheet |
| • | Flow Table Sheet |
| • | Global Specs Sheet |
| • | Pattern Subroutine Sheet |
| • | Pattern Sets Sheet |
| • | Mixed-Signal Timing Sheet |

- Wave Definitions Sheet

Pin Map Sheet

Pin Map		
Group Name	Pin Name	Type
bit0	bit0_p	I/O
	bit0_n	I/O
	bit0_p	Differential
	bit0_n	Differential
Capture	bit0	Differential

Channel Map Sheet

Channel Map				
DIB ID:		View Mode: Pogo		
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
bit0_p		I/O	6.n402	
bit0_n		I/O	6.n406	

Time Sets (Basic) Sheet

Time Sets (Basic)

TERADYNE

Timing Mode: Dual Master Timeset Name:

Time Domain:

Time Set	Cycle		Pin/Group		Data		Drive				Compare			Edge Resolution
	Period	Name	Clock Period	Setup	Src	Fmt	First	Return First	Second	Off	Mode	First	Second	Mode
ts0	40. E-09	Source		2X	PAT	NR-2X	10. E-09		30. E-09	Off		15. E-09	35. E-09	Machine

Pin Levels Sheet

Pin/Group	Seq.	Parameter	Value
Source		Vicm	1.5

Test Instances Sheet

Test Instances			
Test Name		Test Procedure	
		Type	Called As
SrcVBT1		VBT	SrcVBT

The test instance named SrcVBT1 has the following parameters and values as shown in the Instance Editor:

Instance Editor [VBT Test 'SrcVBT1']

Instance: SrcVBT1

Parameters | Timing and Levels | Limits

Time Set: TimeSets 2X

Edge Set:

Pin Levels: Levels

Mixed-Sig:

Overlay:

AC Selector:

AC Category:

DC Selector:

DC Category:

Flow Table Sheet

Flow Table

Label	Enable	Gate			Command	
		Job	Part	Env	Opcode	Parameter
					Test	SrcVBT1
					stop	

Global Specs Sheet

Global Specs

Symbol	Job	Value	Comment
Vcl_default		- 1.E+00	Detector clamp voltage low
Vch_default		6.E+00	Detector clamp voltage high
Data_rate		25.E+06	

Pattern Subroutine Sheet

This sheet is not required because this example does not use pattern subroutines.

Pattern Sets Sheet

This sheet is not required because this example does not use pattern sets.

Mixed-Signal Timing Sheet

This sheet is not required because this example does not use mixed-signal timing sets.

Wave Definitions Sheet

This sheet is not required because this example does not use wave definitions.

<

>

2015 Teradyne, Inc. All rights reserved.

dsscexamples.5.04

<

>

DSSC Examples : [DSSC Dual 2X Serial Source Test](#) : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

DSSC_SOURCE_SERIAL_2X.pat - DSSC_serial						
TSet	Command	Label	Vector /cycle	Source (DSSC-source)	Source	Comment
ts0			0		00	
ts0	set_msb 1048576		1	Start SourceSignal	00	
ts0			2		00	
ts0	set c1 1048576		3		00	
ts0			4		00	
ts0	pipe_minus 0		5		00	
ts0			6		00	
ts0	loop c1		7		00	
ts0		loop:	8	Send	00	
ts0	end_loop loop		9		00	
ts0			10		00	
ts0	halt		11		00	

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.05

	
---	---

[DSSC Examples](#) : [DSSC Dual 2X Serial Source Test](#) : VBT Code

VBT Code

```
Public Function SrcVBT(SourcePins As PinList, SignalName As String, _
    PatternName As Pattern) As Long
    Dim SrcDspWave As New DSPWave
    Dim i As Long
    ' Load levels and timing
    Call TheHdw.Digital.ApplyLevelsTiming(True, True, True)
    ' Set up digsrc
    Call SrcDspWave.CreateConstant(0, 65536)
    ' Create Sinewave Samples to Source
    For i = 0 To 65536 - 1
        SrcDspWave.Element(i) = (65536 - 1) * (Cos(3 * i * 2 * Pi / 65536) + 1) / 2
    Next i
    TheHdw.DSSC.Pins(SourcePins).Pattern(PatternName).Source.Signals.Add SignalName
    ' Load Setting and Samples into DSSC Source Memory
    TheExec.WaveDefinitions.CreateWaveDefinition "WaveDef", SrcDspWave, True
    With TheHdw.DSSC.Pins(SourcePins).Pattern(PatternName).Source.Signals(SignalName)
        .WaveDefinitionName = "WaveDef"
        .SampleSize = 65536
        .SampleRate = 40000000
        .CycleCount = 1
        .Amplitude = 1
        .Offset = 0
        .LoadSettings
```


.LoadSamples

End With

' Start pattern

TheHdw.StartStopwatch

Call TheHdw.Patterns(PatternName).Start("", "")

' Wait for pattern halt

TheHdw.Digital.Patgen.HaltWait

End Function

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dsscexamples.5.06

<	>
-----------------	-----------------

DSSC Examples : [DSSC Dual 2X Serial Source Test](#) : DSP Code

DSP Code

This example does not use DSP code.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.07

<	>
-----------------	-----------------

DSSC Examples : DSSC Dual 2X Serial Capture Test

DSSC Dual 2X Serial Capture Test

The DSSC dual 2X serial capture test captures a signal at dual 2X speeds using VBT to start the pattern and perform the capture.

The HSD1000 board is installed in slot 6.

This workbook example includes the following program components:

- | | |
|---|--------------------------------|
| • | DataTool Sheets |
| • | HSD Pattern with TSet and Pins |
| • | VBT Code |
| • | DSP Code |

<	>
-----------------	-----------------

dsscexamples.5.08

<	>
-----------------	-----------------

DSSC Examples : [DSSC Dual 2X Serial Capture Test](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|---|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Time Sets (Basic) Sheet |
| • | Pin Levels Sheet |
| • | Test Instances Sheet |
| • | Flow Table Sheet |
| • | Global Specs Sheet |
| • | Pattern Subroutine Sheet |
| • | Pattern Sets Sheet |
| • | Mixed-Signal Timing Sheet |

- Wave Definitions Sheet

Pin Map Sheet

Pin Map		
Group Name	Pin Name	Type
bit0	bit0_p	I/O
	bit0_n	I/O
	bit0_p	Differential
	bit0_n	Differential
Capture	bit0	Differential

Channel Map Sheet

Channel Map				
DIB ID:		View Mode: Pogo		
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
bit0_p		I/O	6.n402	
bit0_n		I/O	6.n406	

Time Sets (Basic) Sheet

This sheet is not required because this example does not use basic time sets.

Pin Levels Sheet

Pin Levels			
Pin/Group	Seq.	Parameter	Value
Capture		Vicm	1.5

Test Instances Sheet

Test Instances			
Test Name	Test Procedure		
	Type	Name	Called As
CapVBT1	VBT	CapVBT	

The test instance named CapVBT1 has the following parameters and values as shown in the Instance Editor:

Instance Editor [VBT Test 'CapVBT']

Instance: CapVBT1

Parameters

Timing and Levels

Limits

Time Set: TimeSets 2X

Edge Set:

Pin Levels: Levels

Mixed-Sig:

Overlay:

AC Selector:

AC Category:

DC Selector:

DC Category:

Flow Table Sheet

Flow Table

Label	Enable	Job	Gate			Command	
			Part	Env		Opcode	Parameter
						Test	CapVBT1
						stop	

Global Specs Sheet

Global Specs

Symbol	Job	Value	Comment
Vcl_default		- 1.E+00	Detector clamp voltage low
Vch_default		6.E+00	Detector clamp voltage high
Data_rate		25.E+06	

Pattern Subroutine Sheet

This sheet is not required because this example does not use pattern subroutines.

Pattern Sets Sheet

This sheet is not required because this example does not use pattern sets.

Mixed-Signal Timing Sheet

This sheet is not required because this example does not use mixed-signal timing sets.

Wave Definitions Sheet

This sheet is not required because this example does not use wave definitions.

<

>

2015 Teradyne, Inc. All rights reserved.

dsscexamples.5.09

<

>

DSSC Examples : [DSSC Dual 2X Serial Capture Test](#) : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

DSSC_2X_SERIAL_CAPTURE.pat - DSSC_CAP_SERIAL						
TSet	Command	Label	Vector /cycle	Capture (DSSC-Capture)	Capture	Comment
ts0			0		00	
ts0	set_msb 1048576		1		00	
ts0			2		00	
ts0	set c1 1048576		3		00	
ts0			4		00	
ts0	pipe_minus 0		5		00	
ts0			6		00	
ts0	loop c1		7		00	
ts0		loop:	8		00	
ts0	end_loop loop		9	Store	VV	
ts0			10		00	
ts0	halt		11		00	

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.10

	
---	---

DSSC Examples : DSSC Dual 2X Serial Capture Test : VBT Code

VBT Code

```
Public Function CapVBT(CapturePins As PinList, SignalName As String, _
    PatternName As Pattern) As Long
    Dim CapDspWave As New DSPWave
    Dim i As Long
    ' Load levels and timing
    Call TheHdw.Digital.ApplyLevelsTiming(True, True, True)
    ' Set up digcap
    TheHdw.DSSC.Pins(CapturePins).Pattern(PatternName).Capture.Signals.Add SignalName
    With TheHdw.DSSC.Pins(CapturePins).Pattern(PatternName).Capture.Signals(SignalName)
        .SampleSize = 65536
        .SampleRate = 40000000
        .CycleCount = 1
        .LoadSettings
    End With
    ' Start pattern
    TheHdw.StartStopwatch
    Call TheHdw.Patterns(PatternName).Start("", "")
    ' Wait for pattern halt
    TheHdw.Digital.Patgen.HaltWait
    ' Return capture dspwaves
    CapDspWave = TheHdw.DSSC(CapturePins).Pattern(PatternName).Capture.Signals( _
        SignalName).DSPWave.Pins(CapturePins).Value(0)
    CapDspWave.Plot (PatternName)
```

End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dsscexamples.5.11

<	>
-----------------	-----------------

DSSC Examples : [DSSC Dual 2X Serial Capture Test](#) : DSP Code

DSP Code

[This example does not use DSP code.](#)

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.12

<	>
-----------------	-----------------

DSSC Examples : Using Default Signals with DSSC

Using Default Signals with DSSC

The IG-XL default signal feature allows you to write a generic pattern that you can use to source different waveforms. This example defines three signals, assigns a default signal name, and bursts the same pattern once for each of the three signals.

The HSD1000 board is installed in slot 8.

This workbook example includes the following program components:

- | | |
|---|--------------------------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | HSD Pattern with TSet and Pins |
| • | VBT Code |
| • | DSP Code |

<	>
-----------------	-----------------

dsscexamples.5.13

<	>
-----------------	-----------------

DSSC Examples : [Using Default Signals with DSSC](#) : DUT Connections

DUT Connections

This example uses the DSSC's ability to use source and capture functions on the same pin and therefore does not require a device.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.14

<	>
-----------------	-----------------

DSSC Examples : Using Default Signals with DSSC : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|---------------------------|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Time Sets (Basic) Sheet |
| • | Pin Levels Sheet |
| • | Test Instances Sheet |
| • | Flow Table Sheet |
| • | Global Specs Sheet |
| • | Pattern Subroutine Sheet |
| • | Pattern Sets Sheet |
| • | Mixed-Signal Timing Sheet |

- [Wave Definitions Sheet](#)

Pin Map Sheet

Pin Map

Group Name	Pin Name	Type
All_pins	bit0	I/O
	bit1	I/O
	bit1	I/O
	bit0	I/O

Channel Map Sheet

Channel Map

DIB ID:

View Mode:

Device Under Test		Tester Channel		Comment
Pin Name	Package Pin	Type	Site 0	
bit0		I/O	8.n402	
bit1		I/O	8.n406	

Time Sets (Basic) Sheet

Time Sets (Basic)

Timing Mode: Master Timeset Name:

Time Domain:

	Cycle	Pin/Group	Data	Drive	Compare	Edge Resolution				
Time Set	Period	Name 	Clock Period Setup	Src Fmt On	Data Return Off	Mode Open Close Mode				
ts0	50. E-09	All pins		PAT NR	0	Off				Machine

Pin Levels Sheet

Pin Levels

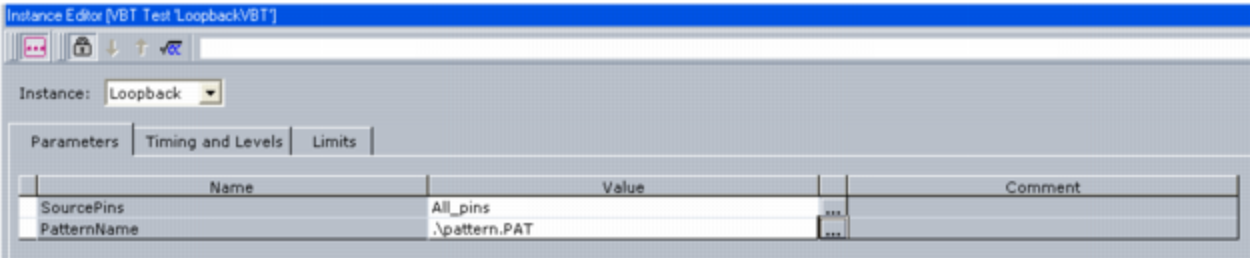
Pin/Group	Seq.	Parameter	Value
All_pins		Vil	0
All_pins		Vih	3
All_pins		Vol	1
All_pins		Voh	2
All_pins		Vt	1.5
All_pins		Vcl	-1
All_pins		Vch	6
All_pins		VoutLoTyp	0
All_pins		VoutHiTyp	3
All_pins		DriverMode	Largeswing-VT

Test Instances Sheet

Test Instances

	Test Procedure			DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
Loopback	VBT	LoopbackVBT						TSB		Levels		

The test instance named Loopback has the following parameters and values as shown in the Instance Editor:



Flow Table Sheet

Command			
Opcode	Parameter	TName	TNum
Test	Loopback		
stop			

Global Specs Sheet

Global Specs			
Symbol	Job	Value	Comment
Vcl_default		- 1. E+00	Detector clamp voltage low
Vch_default		6. E+00	Detector clamp voltage high
Data_rate		20. E+06	

Pattern Subroutine Sheet

This sheet is not required because this example does not use pattern subroutines.

Pattern Sets Sheet

This sheet is not required because this example does not use pattern sets.

Mixed-Signal Timing Sheet

This sheet is not required because this example does not use mixed-signal timing sets.

Wave Definitions Sheet

This sheet is not required because this example does not use wave definitions.

<

>

2015 Teradyne, Inc. All rights reserved.

dsscexamples.5.15

<

>

DSSC Examples : Using Default Signals with DSSC : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

pattern.pat (pattern_module - SRM)						
				+All_pins		
TSet	Command	Label	Vector /Cycle	All_pins (DSSC-Source)	bb 11 tt 10	Comment
ts0			0	Start	00	
ts0	repeat 144		1		00	
ts0	stv		2	Send	00	
ts0	halt		3		00	

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.16

	
---	---

DSSC Examples : Using Default Signals with DSSC : VBT Code

VBT Code

Public Function LoopbackVBT(SourcePins As PinList, PatternName As Pattern) As Long

' Load levels and timing

Call TheHdw.Digital.ApplyLevelsTiming(False, True, True)

TheHdw.Digital.HRAM.CaptureType = captSTV

' Set up digsrc

Call setup_digsrc(SourcePins, "SignalName1", PatternName, _
"WaveDef1", PatternName, 1)

Call setup_digsrc(SourcePins, "SignalName2", PatternName, _
"WaveDef2", PatternName, 2)

Call setup_digsrc(SourcePins, "SignalName3", PatternName, _
"WaveDef3", PatternName, 3)

TheHdw.DSSC.Pins(SourcePins).Pattern(PatternName).Source.Signals.DefaultSignal _
= "SignalName1"

TheHdw.Patterns(PatternName).Start

TheHdw.Digital.Patgen.HaltWait

MsgBox "Result of first burst = " & _

TheHdw.Digital.HRAM.Pins(SourcePins).PatData(0)

TheHdw.DSSC.Pins(SourcePins).Pattern(PatternName).Source.Signals.DefaultSignal _
= "SignalName2"

TheHdw.Patterns(PatternName).Start

TheHdw.Digital.Patgen.HaltWait

MsgBox "Result of second burst = " & _

TheHdw.Digital.HRAM.Pins(SourcePins).PatData(0)

```

TheHdw.DSSC.Pins(SourcePins).Pattern(PatternName).Source.Signals.DefaultSignal _
    = "SignalName3"
TheHdw.Patterns(PatternName).Start
TheHdw.Digital.Patgen.HaltWait
MsgBox "Result of third burst = " & _
    TheHdw.Digital.HRAM.Pins(SourcePins).PatData(0)

```

End Function

```

Public Function setup_digsrc(SourcePins As PinList, _
    sigName As String, _
    srcPatt As Pattern, _
    WaveDefName As String, _
    PatternName As Pattern, _
    Value As Integer) As Long
    Dim SrcDspWave As New DSPWave
    Call SrcDspWave.CreateConstantInt(Value, 1)
    ' Create signal collection for DigSrc
    TheHdw.DSSC.Pins(SourcePins).Pattern(srcPatt).Source.Signals.Add sigName
    ' Link source data to source waveform
    TheExec.WaveDefinitions.CreateWaveDefinition WaveDefName, SrcDspWave, True
    ' Set up the sample size and the amplitude of the source waveform
    With TheHdw.DSSC.Pins(SourcePins).Pattern(srcPatt).Source.Signals.Item(sigName)
        .WaveDefinitionName = WaveDefName
        .SampleSize = 1
        .SampleRate = 1
        .CycleCount = 1
        .amplitude = 1
        .Offset = 0
        .LoadSettings
        .LoadSamples
    End With
End Function

```

<	>
--	--

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dsscexamples.5.17

<	>
-----------------	-----------------

DSSC Examples : Using Default Signals with DSSC : DSP Code

DSP Code

This example does not use DSP code.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--